



Digital Integrated Circuits

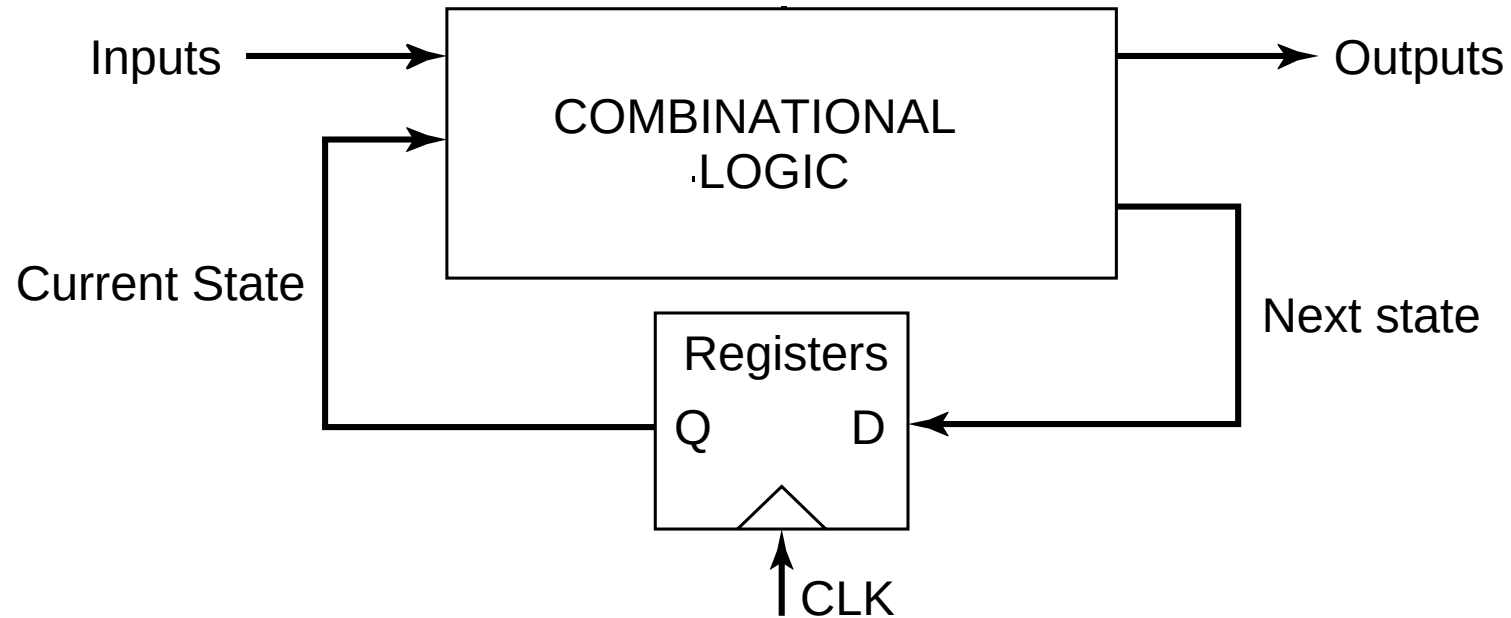
A Design Perspective

Jan M. Rabaey
Anantha Chandrakasan
Borivoje Nikolic

Designing Sequential Logic Circuits

November 2002

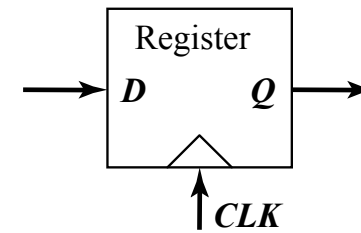
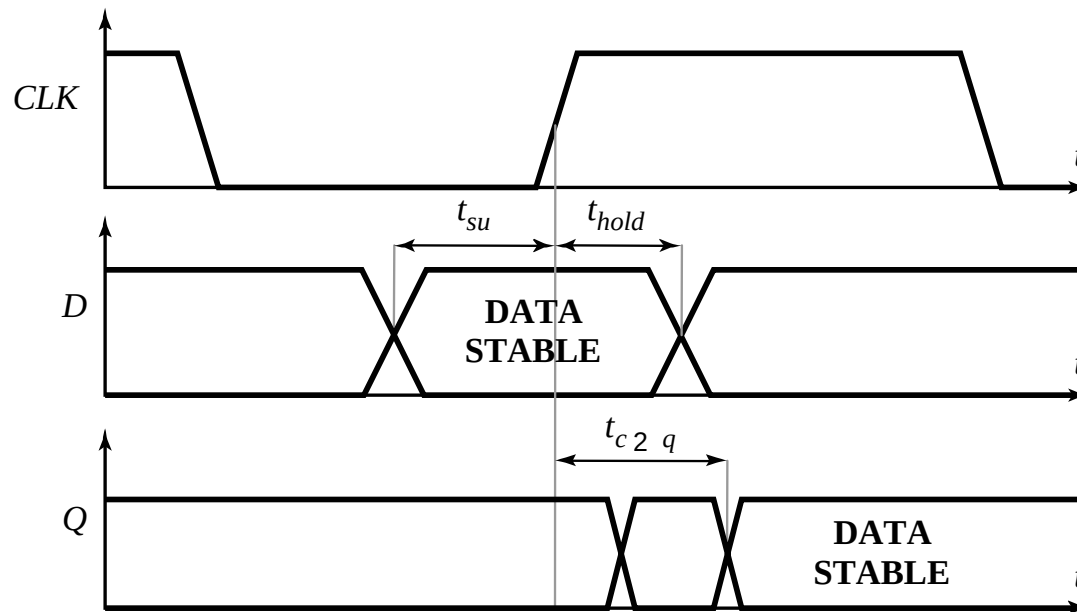
Sequential Logic



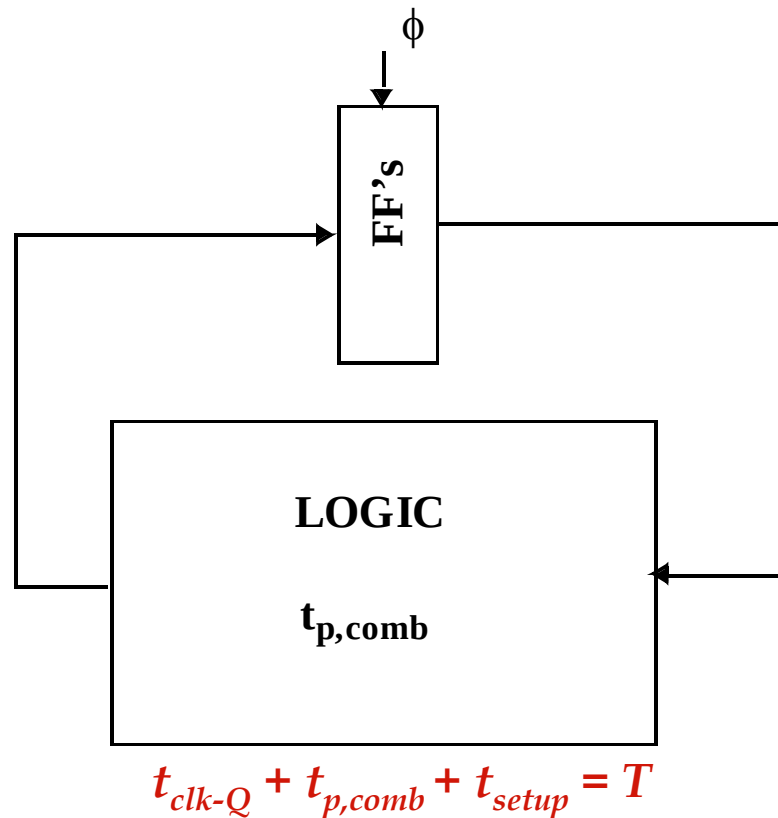
2 storage mechanisms

- positive feedback
- charge-based

Timing Definitions



Maximum Clock Frequency



Also:

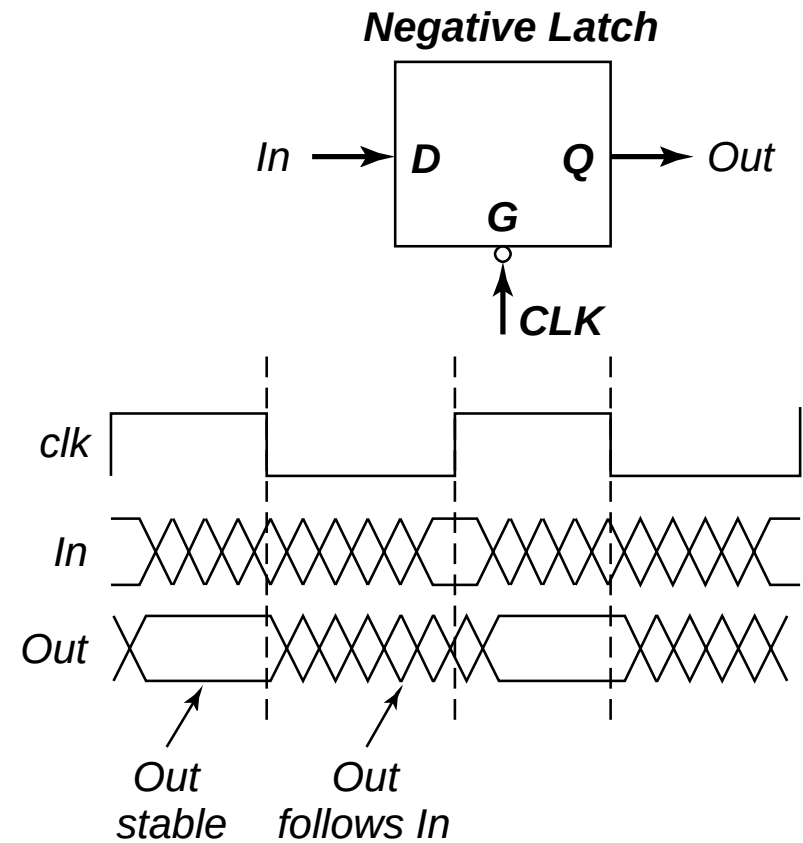
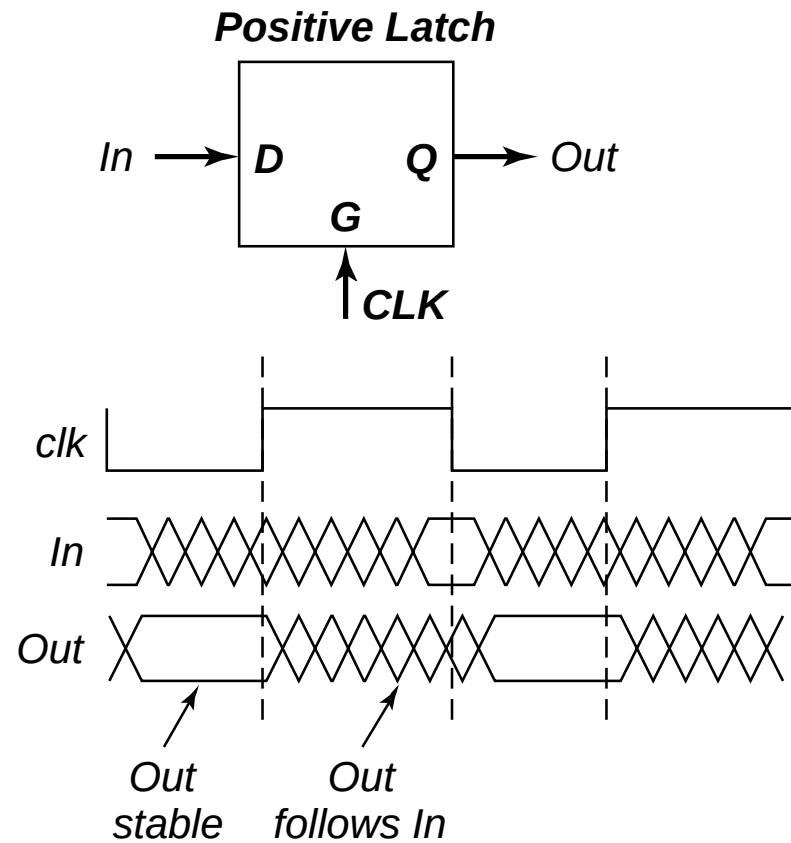
$$t_{cdreg} + t_{cdlogic} > t_{hold}$$

t_{cd} : contamination delay =
minimum delay

Naming Conventions

- In our text:
 - a latch is level sensitive
 - a register is edge-triggered
- There are many different naming conventions
 - For instance, many books call edge-triggered elements flip-flops
 - This leads to confusion however

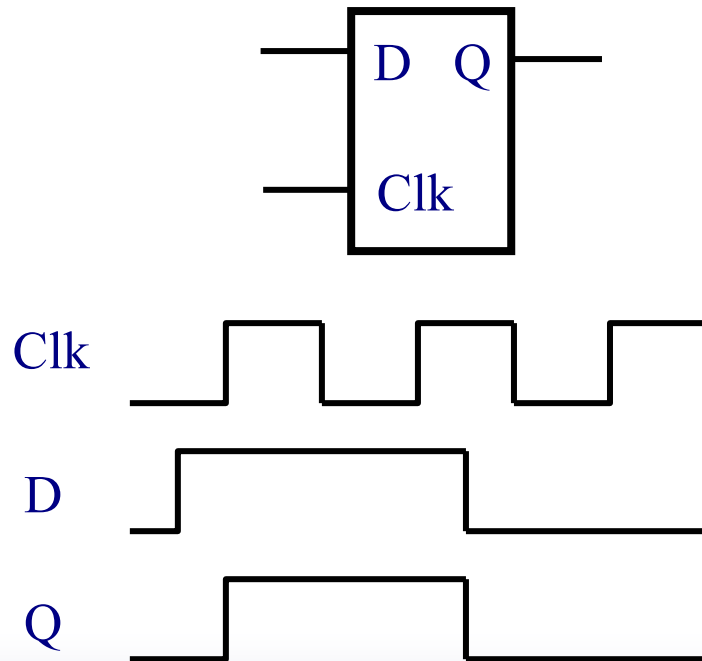
Latches



Latch versus Register

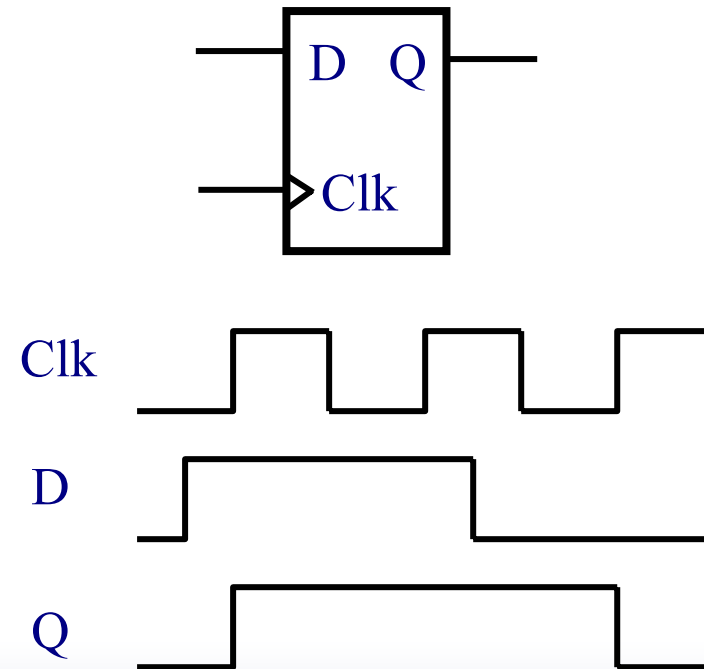
❑ Latch

stores data when
clock is low

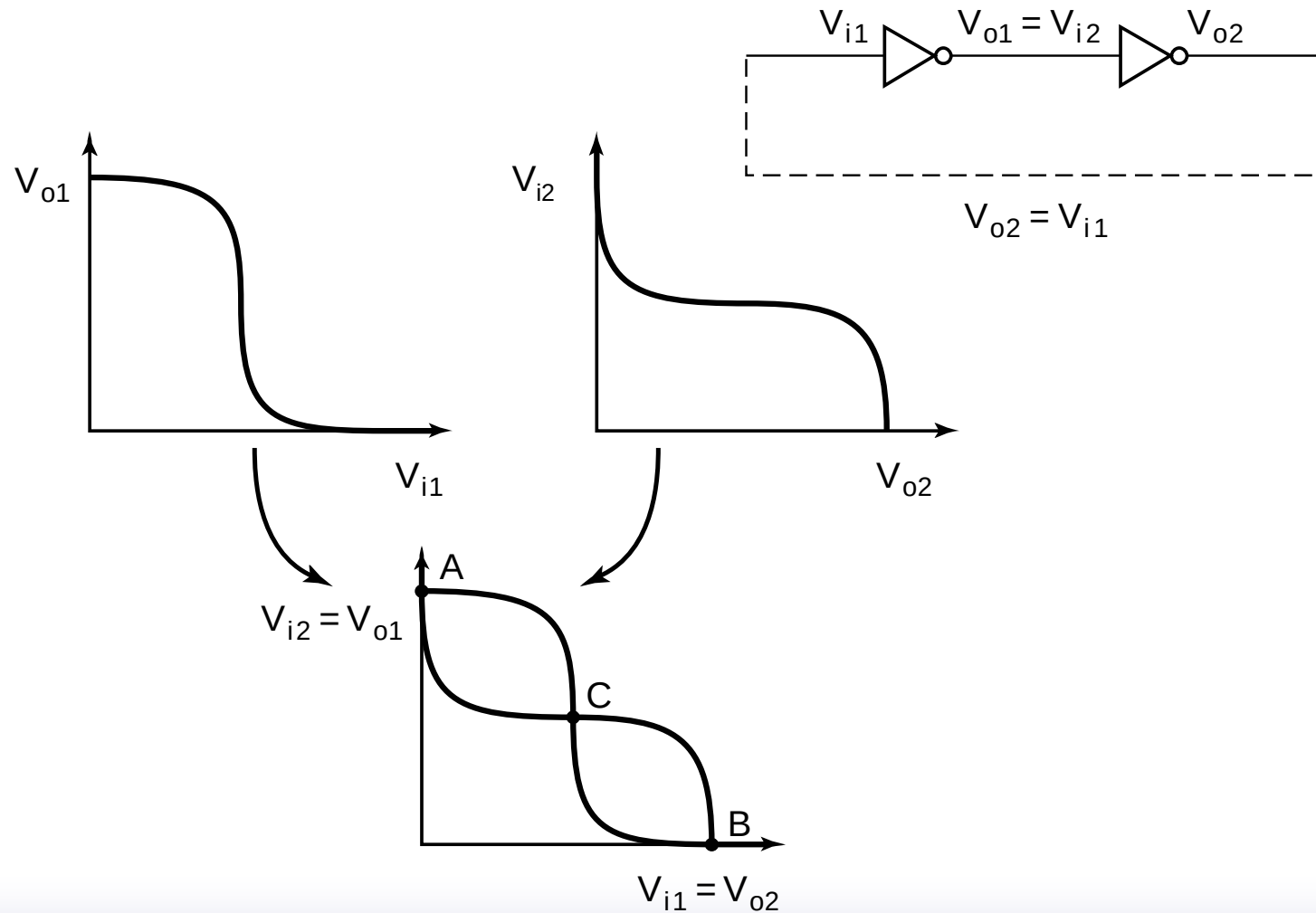


❑ Register

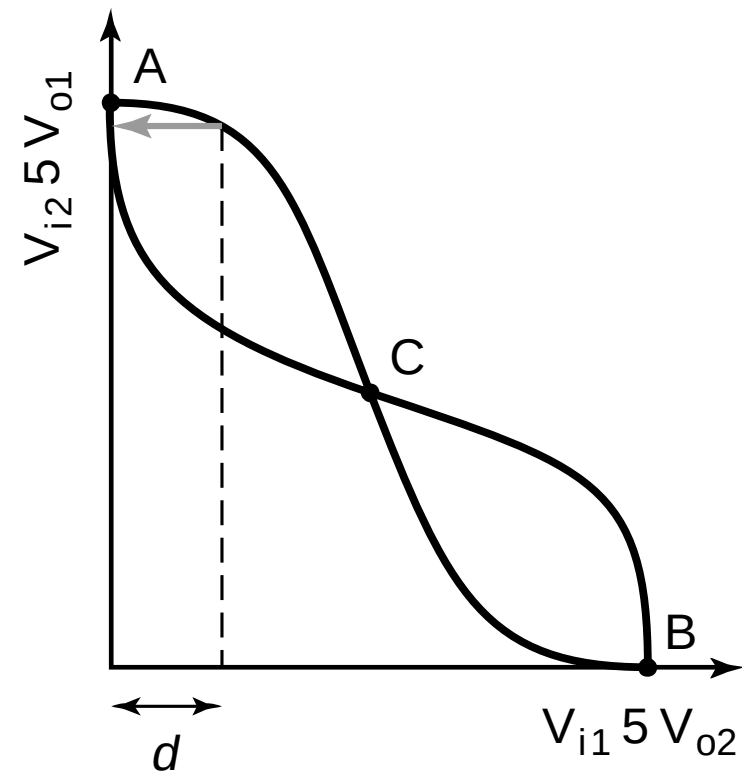
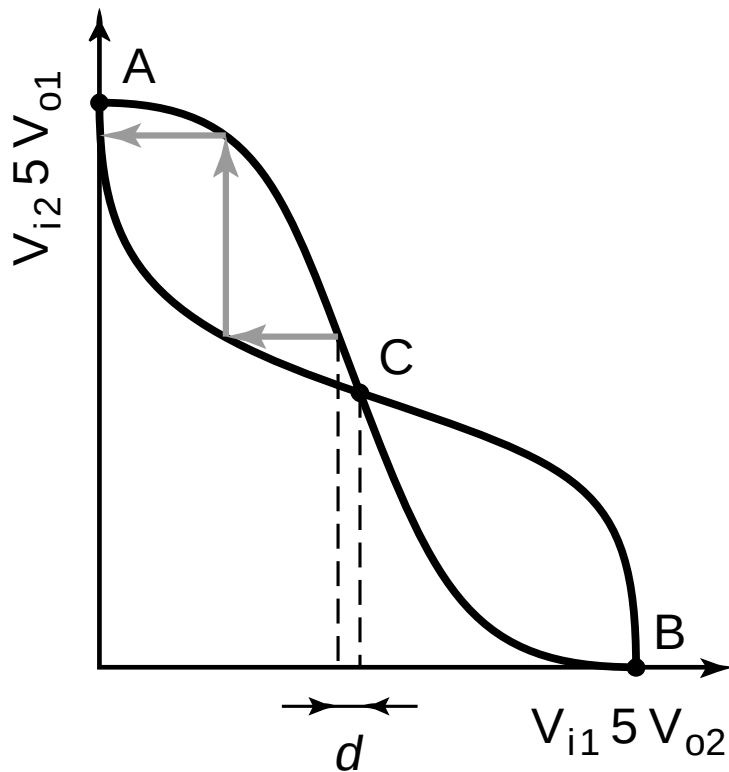
stores data when
clock rises



Positive Feedback: Bi-Stability



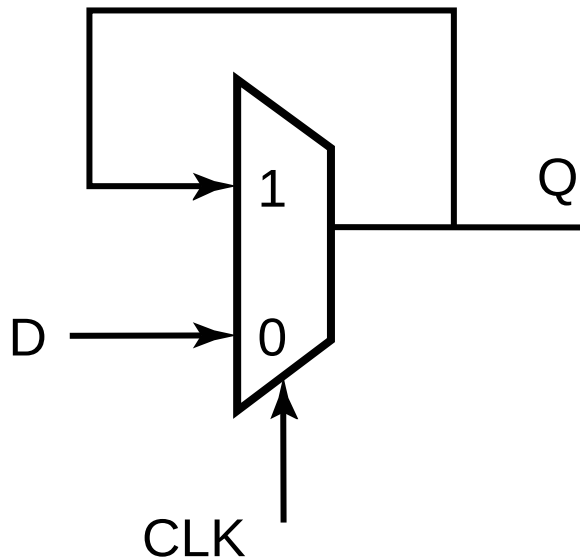
Meta-Stability



Gain should be larger than 1 in the transition region

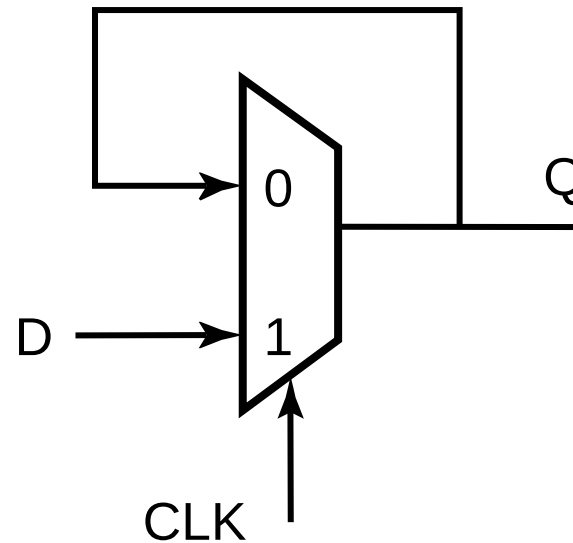
Mux-Based Latches

Negative latch
(transparent when CLK= 0)



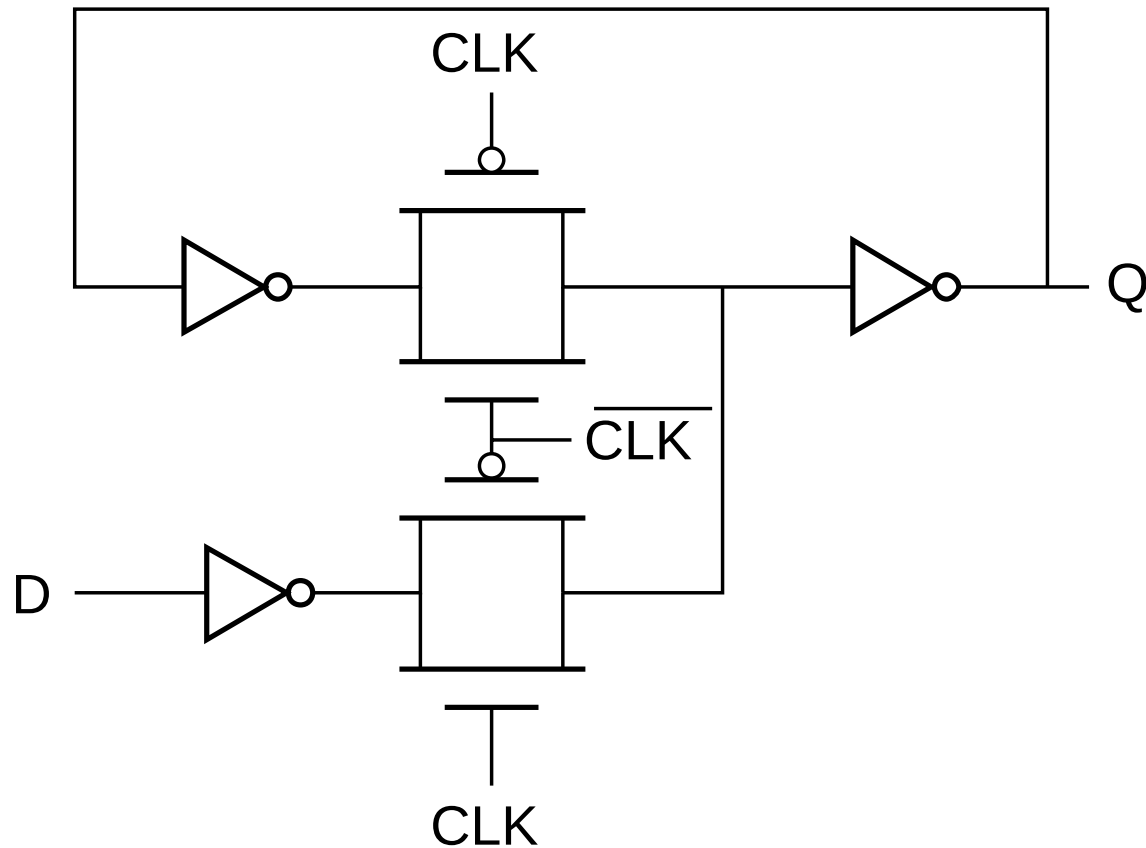
$$Q = Clk \cdot Q + \overline{Clk} \cdot In$$

Positive latch
(transparent when CLK= 1)

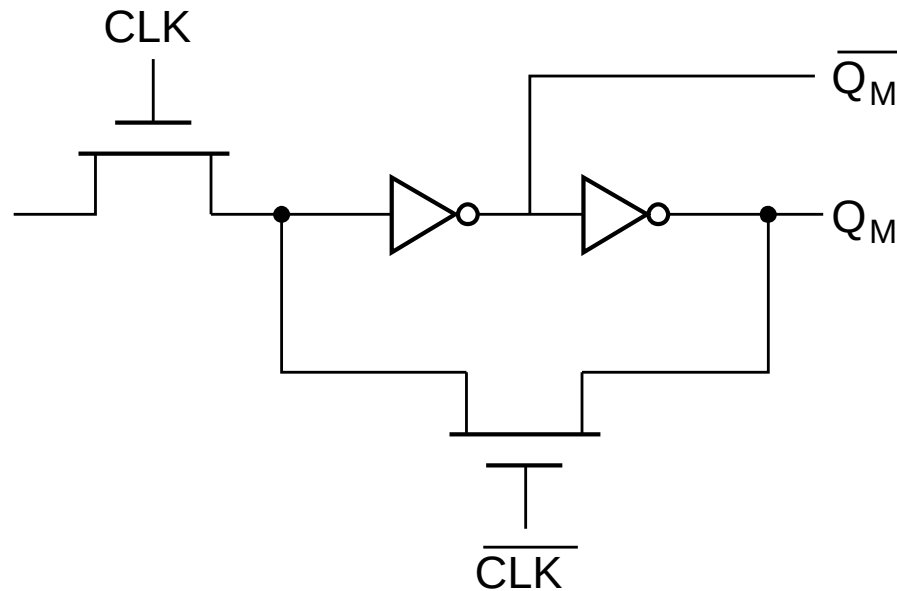


$$Q = \overline{Clk} \cdot Q + Clk \cdot In$$

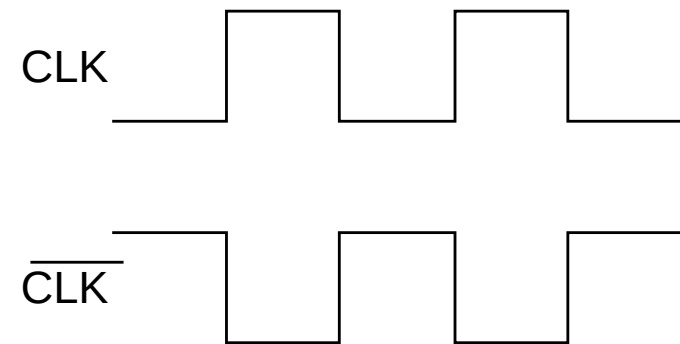
Mux-Based Latch



Mux-Based Latch



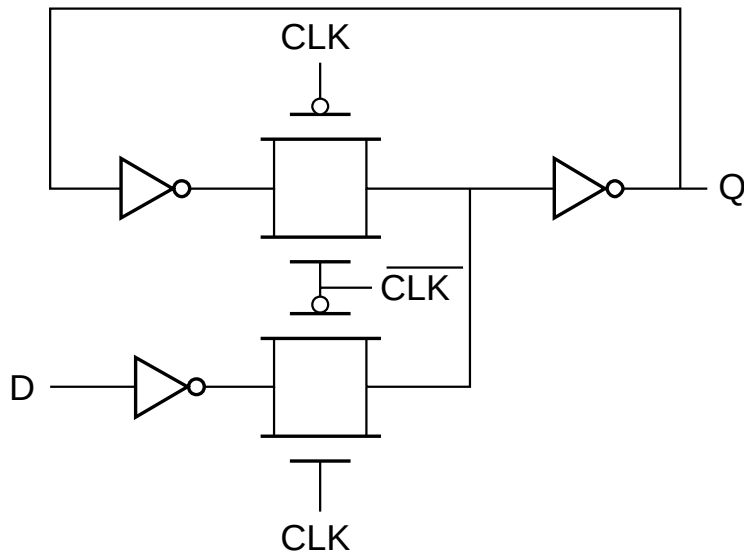
NMOS only



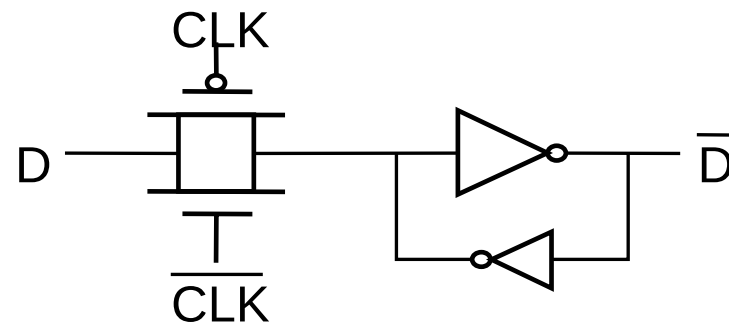
Non-overlapping clocks

Writing into a Static Latch

Use the clock as a decoupling signal,
that distinguishes between the transparent and opaque states

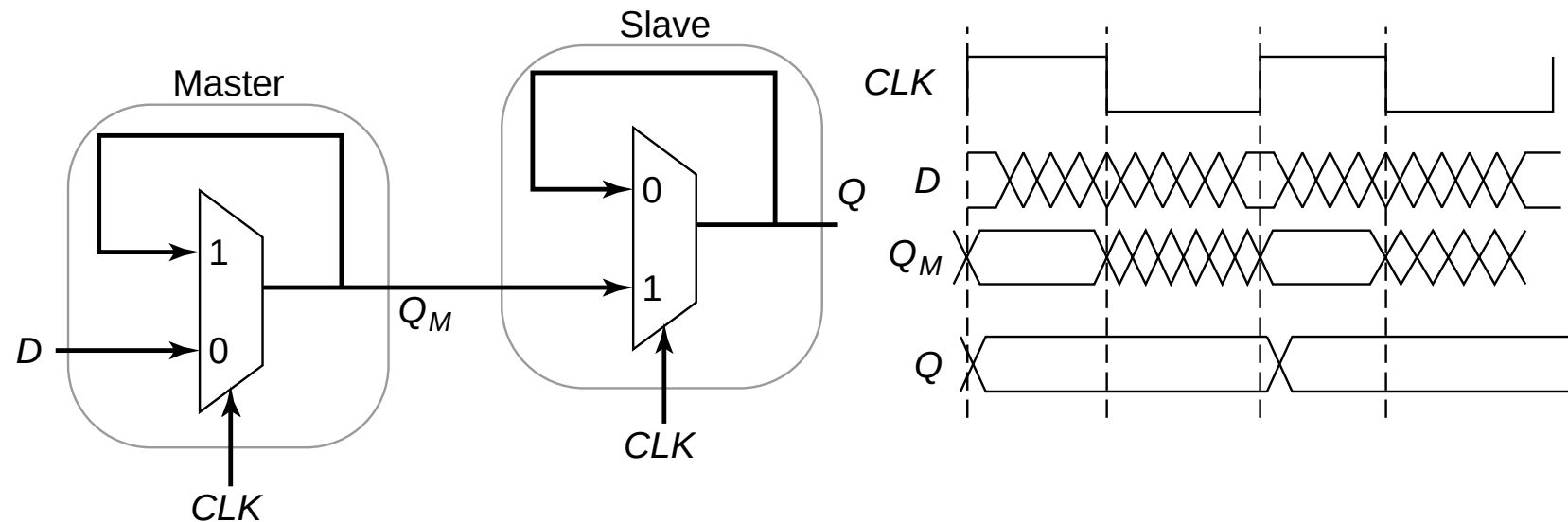


Converting into a MUX



Forcing the state
(can implement as NMOS-only)

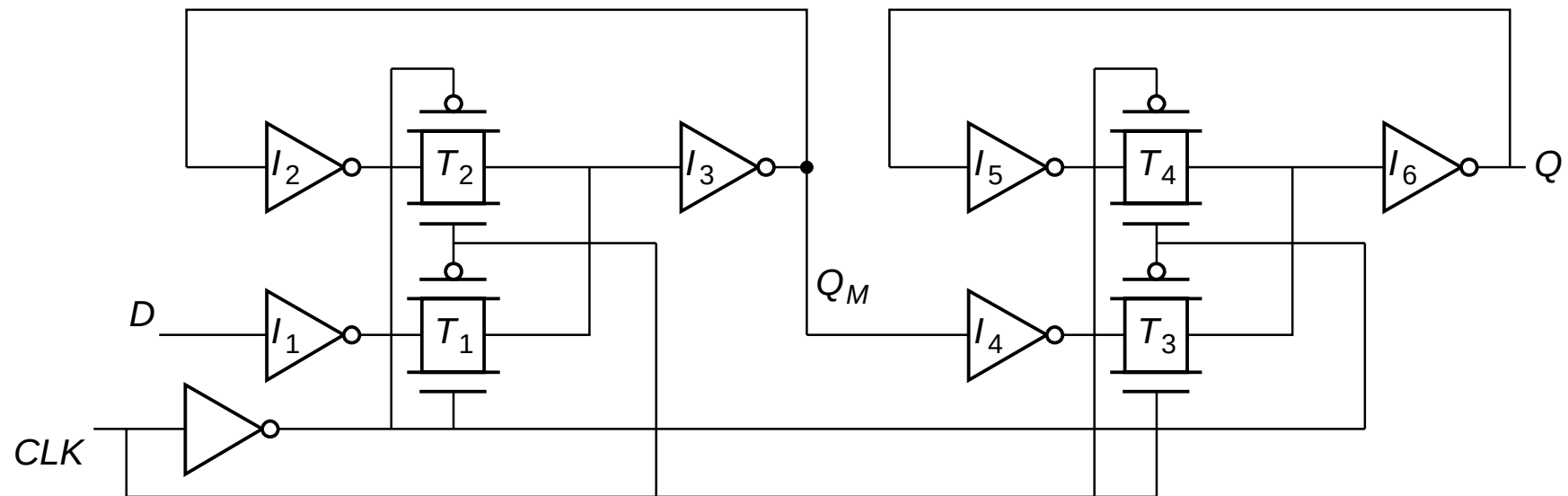
Master-Slave (Edge-Triggered) Register



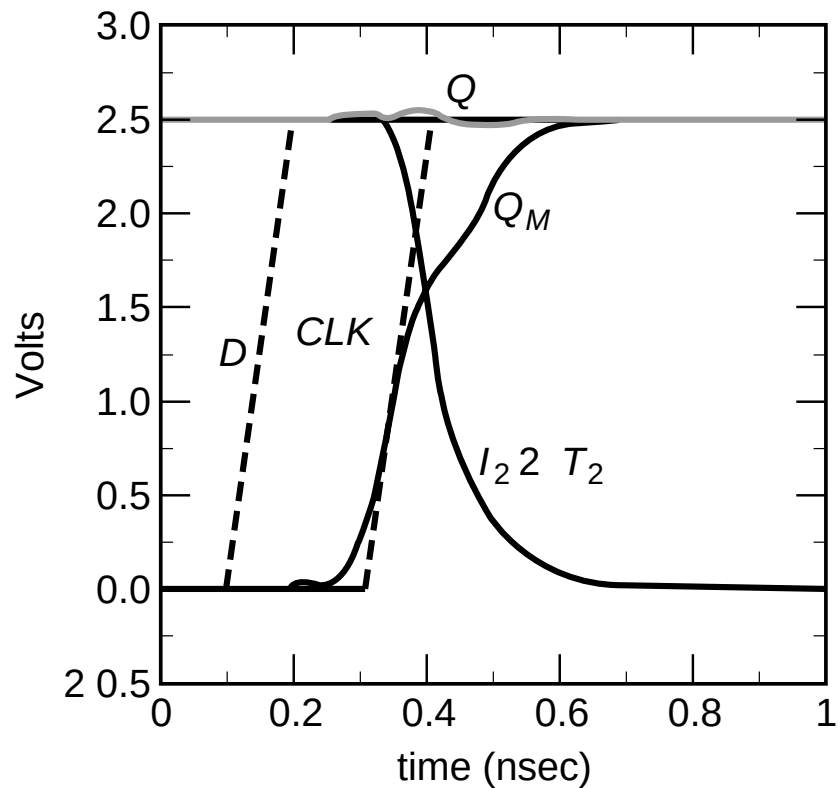
Two opposite latches trigger on edge
Also called master-slave latch pair

Master-Slave Register

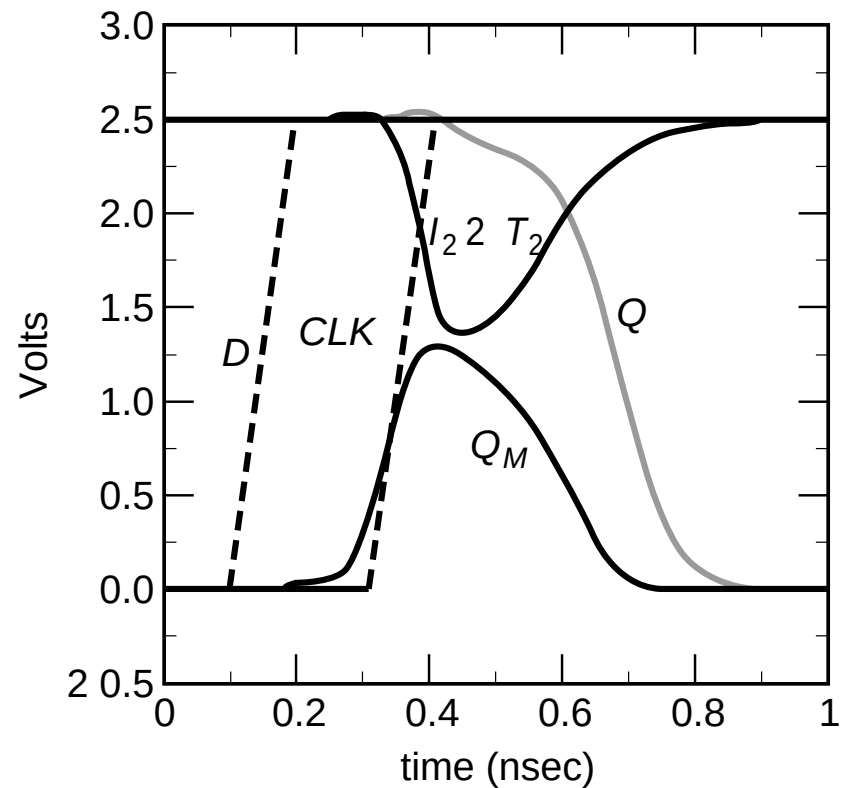
Multiplexer-based latch pair



Setup Time

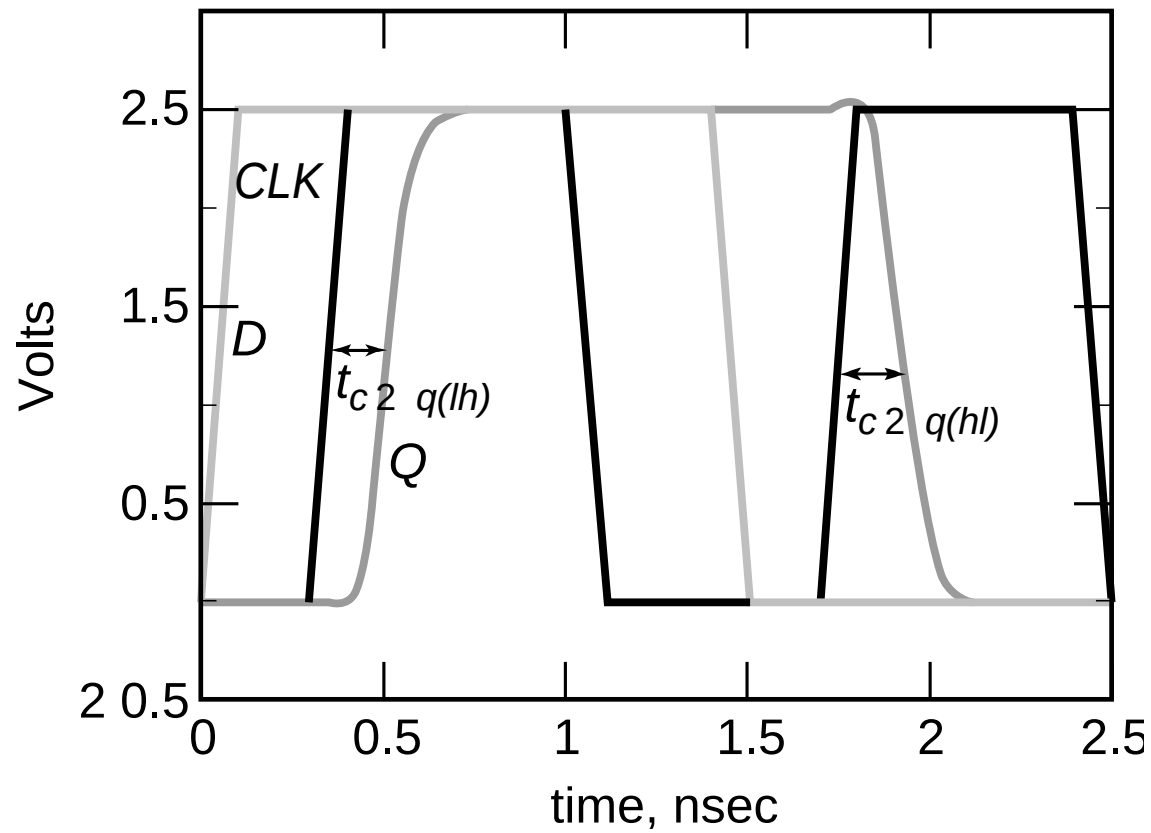


(a) $T_{setup} = 0.21$ nsec

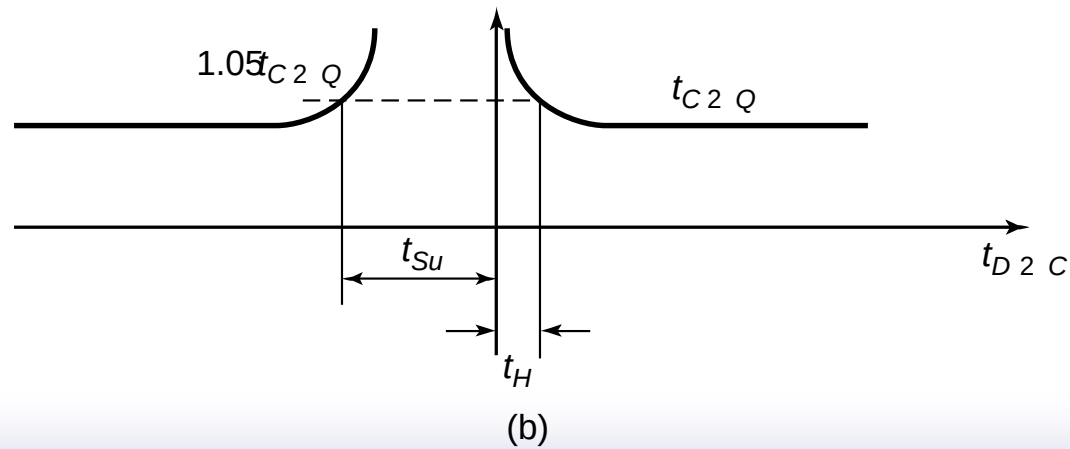
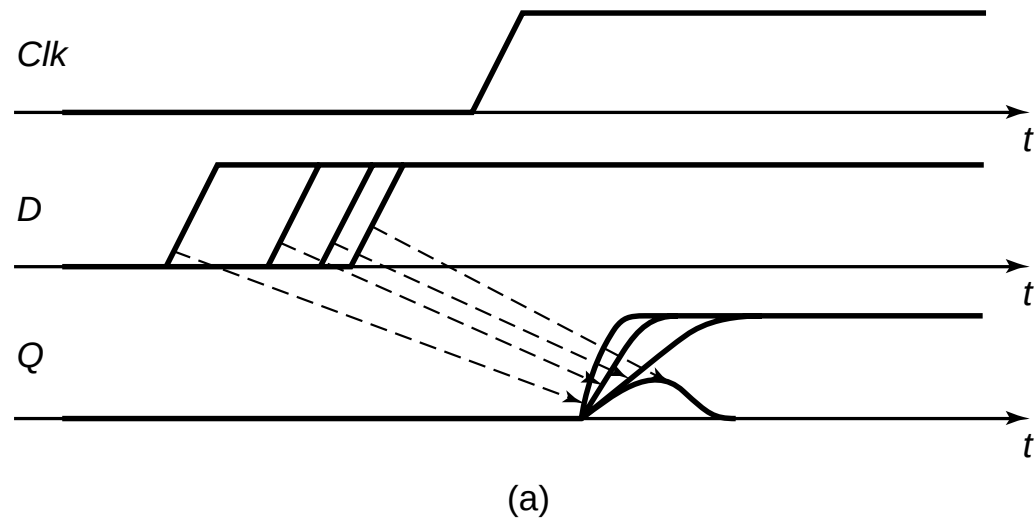


(b) $T_{setup} = 0.20$ nsec

Clk-Q Delay

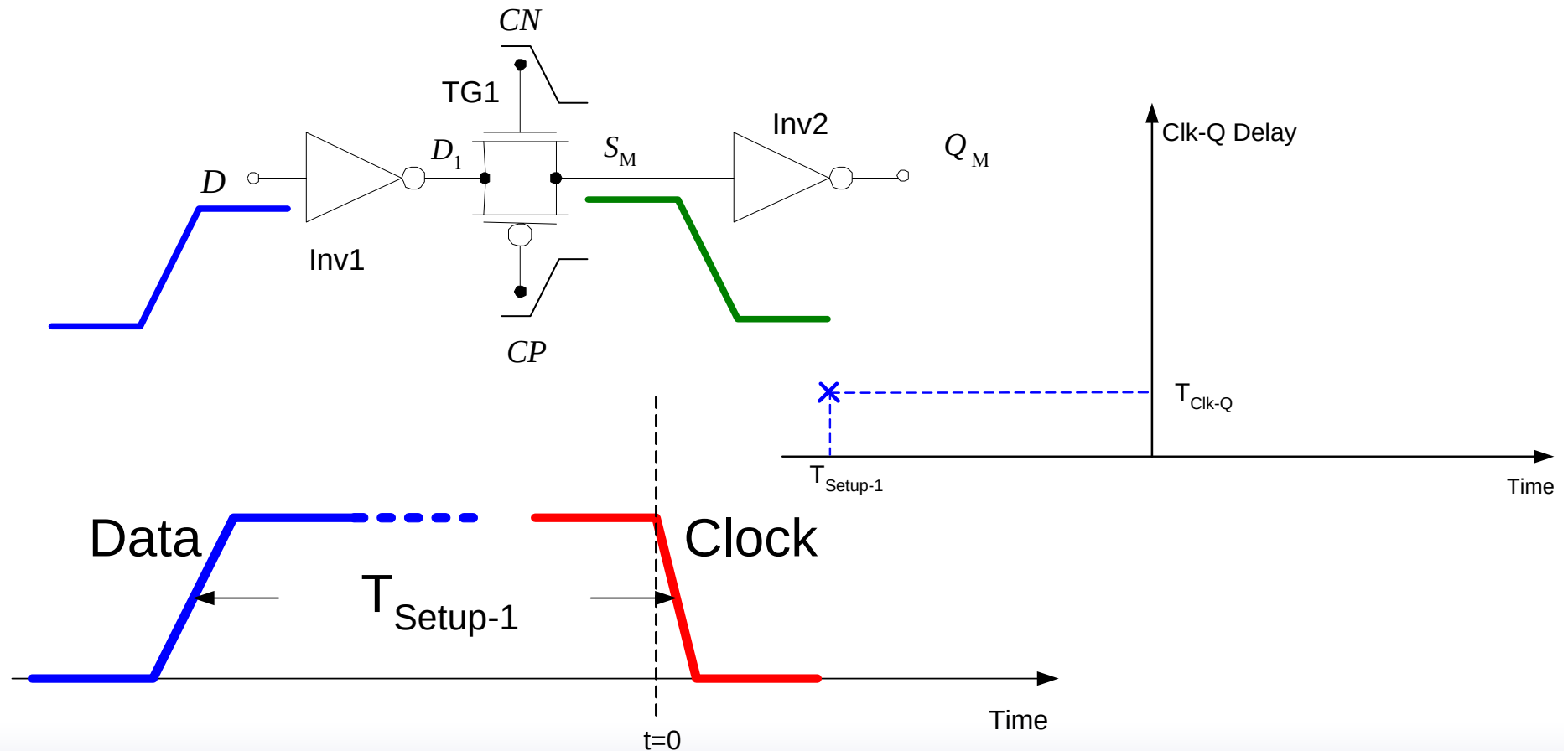


More Precise Setup Time



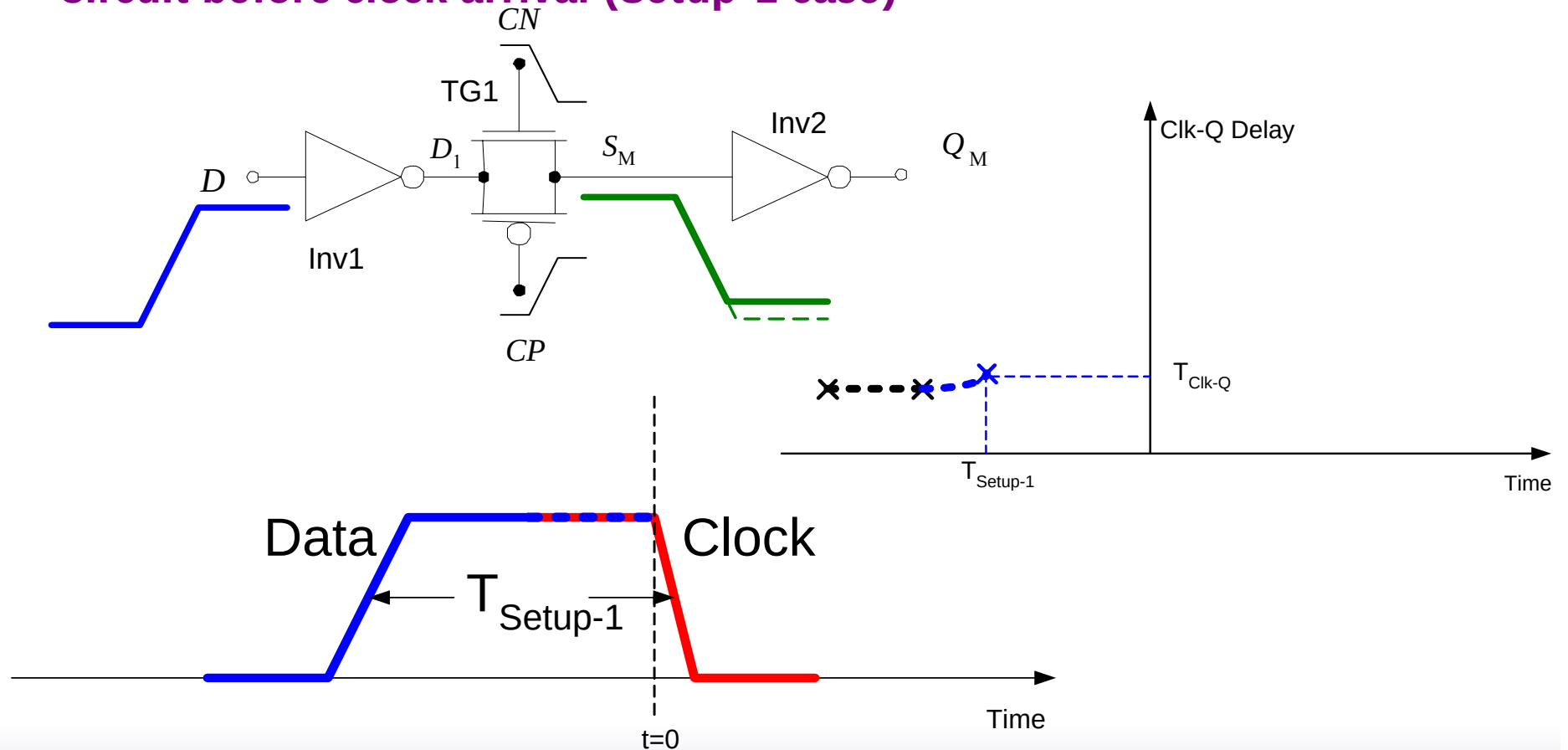
Setup/Hold Time Illustrations

Circuit before clock arrival (Setup-1 case)



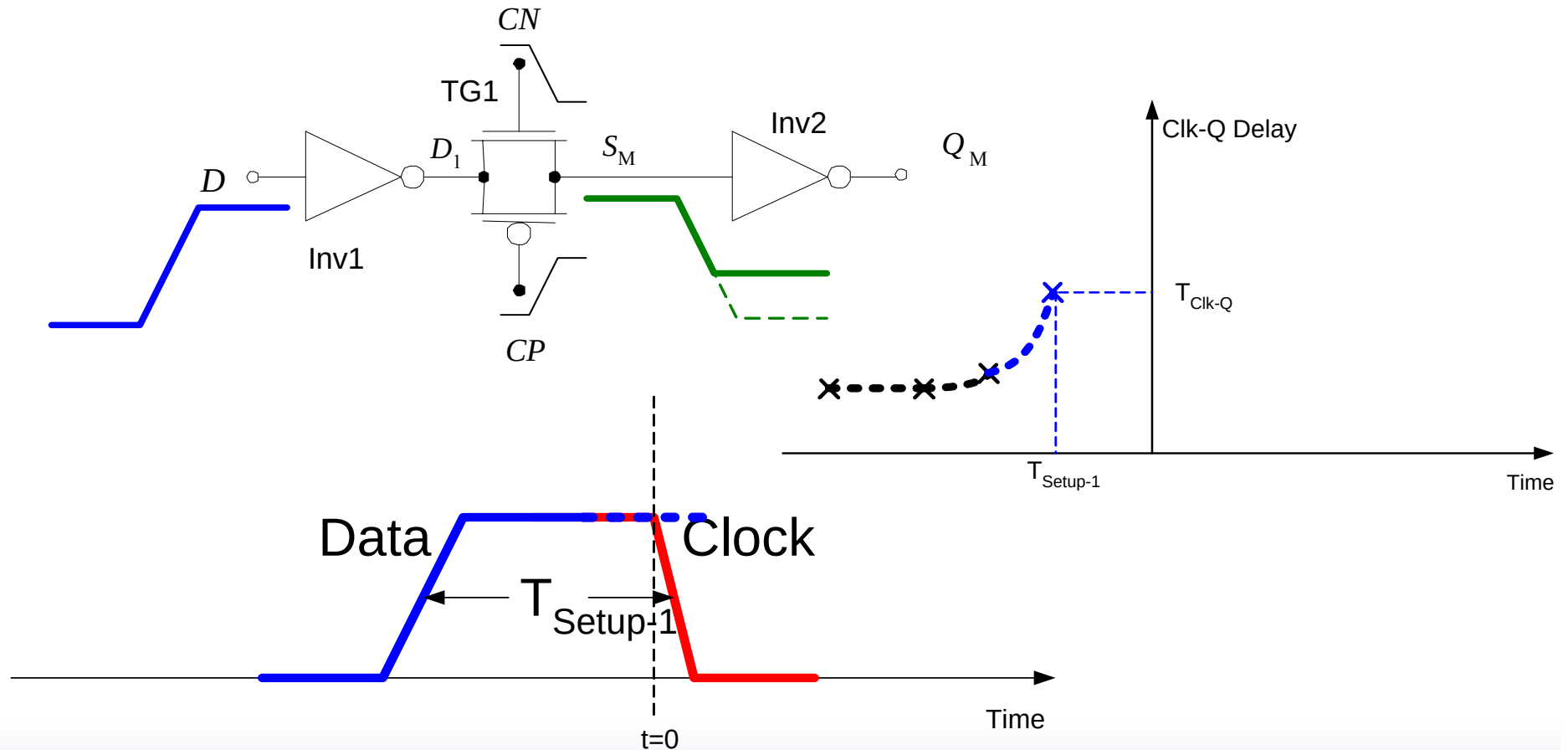
Setup/Hold Time Illustrations

Circuit before clock arrival (Setup-1 case)



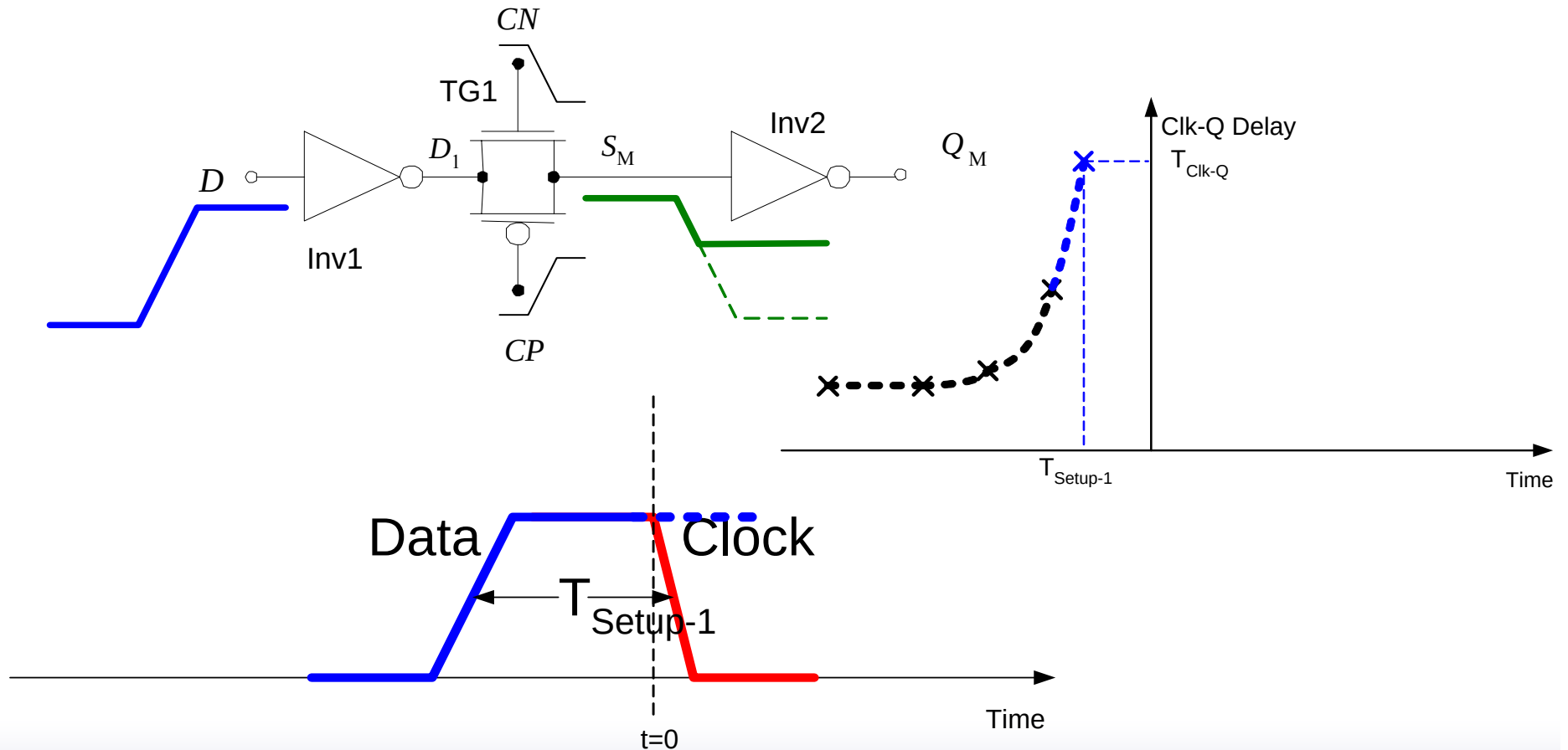
Setup/Hold Time Illustrations

Circuit before clock arrival (Setup-1 case)



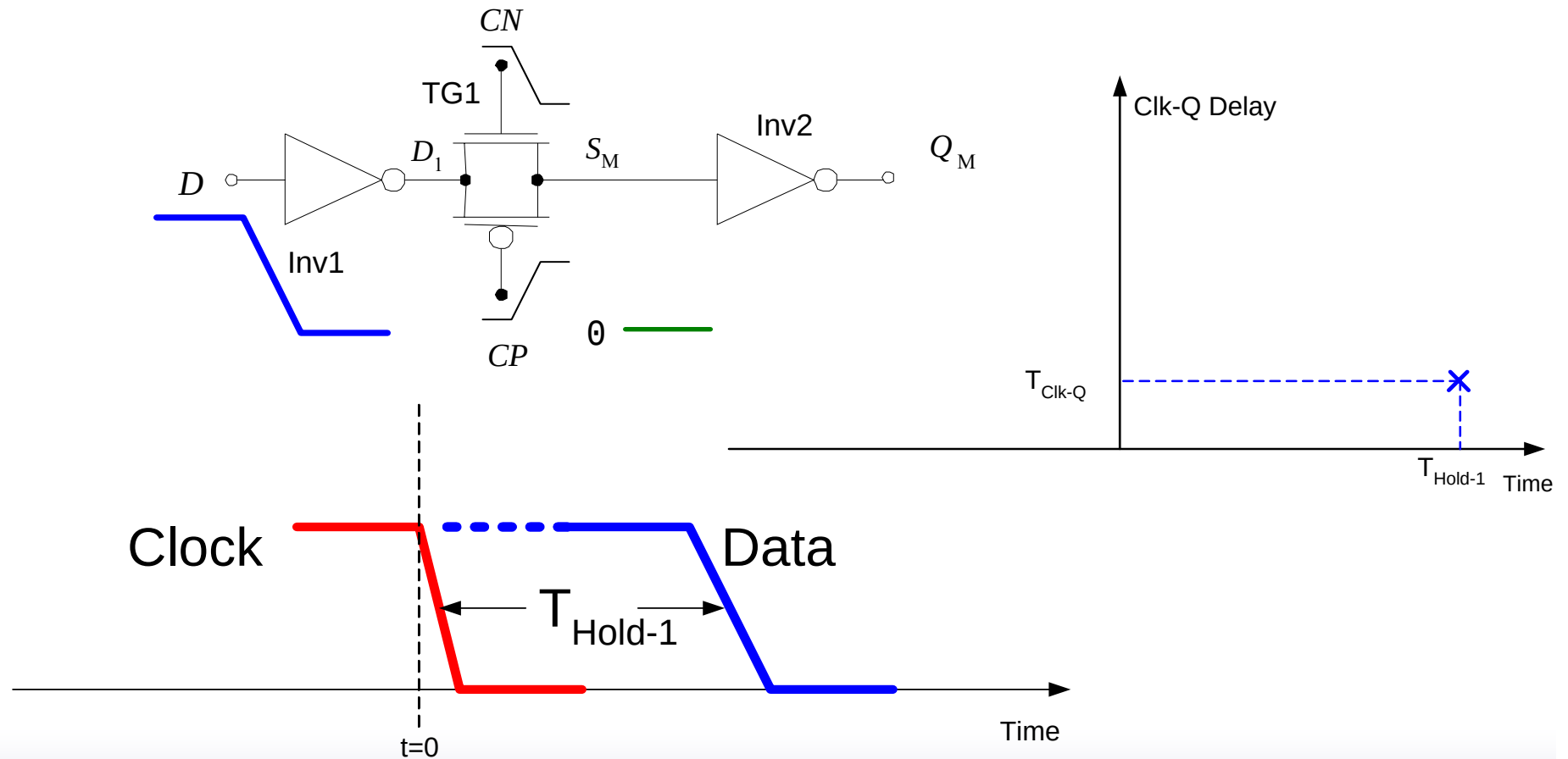
Setup/Hold Time Illustrations

Circuit before clock arrival (Setup-1 case)



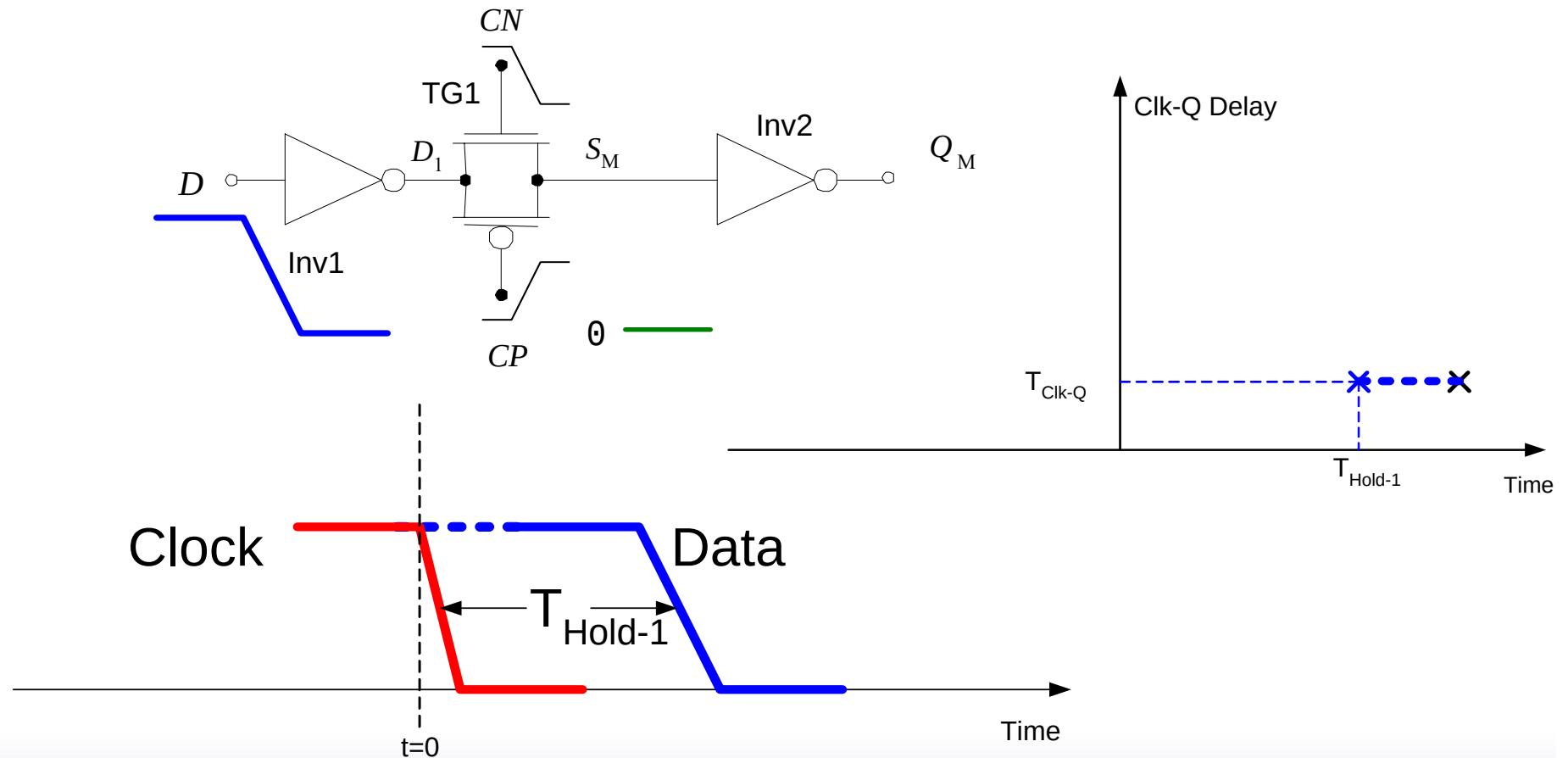
Setup/Hold Time Illustrations

Hold-1 case



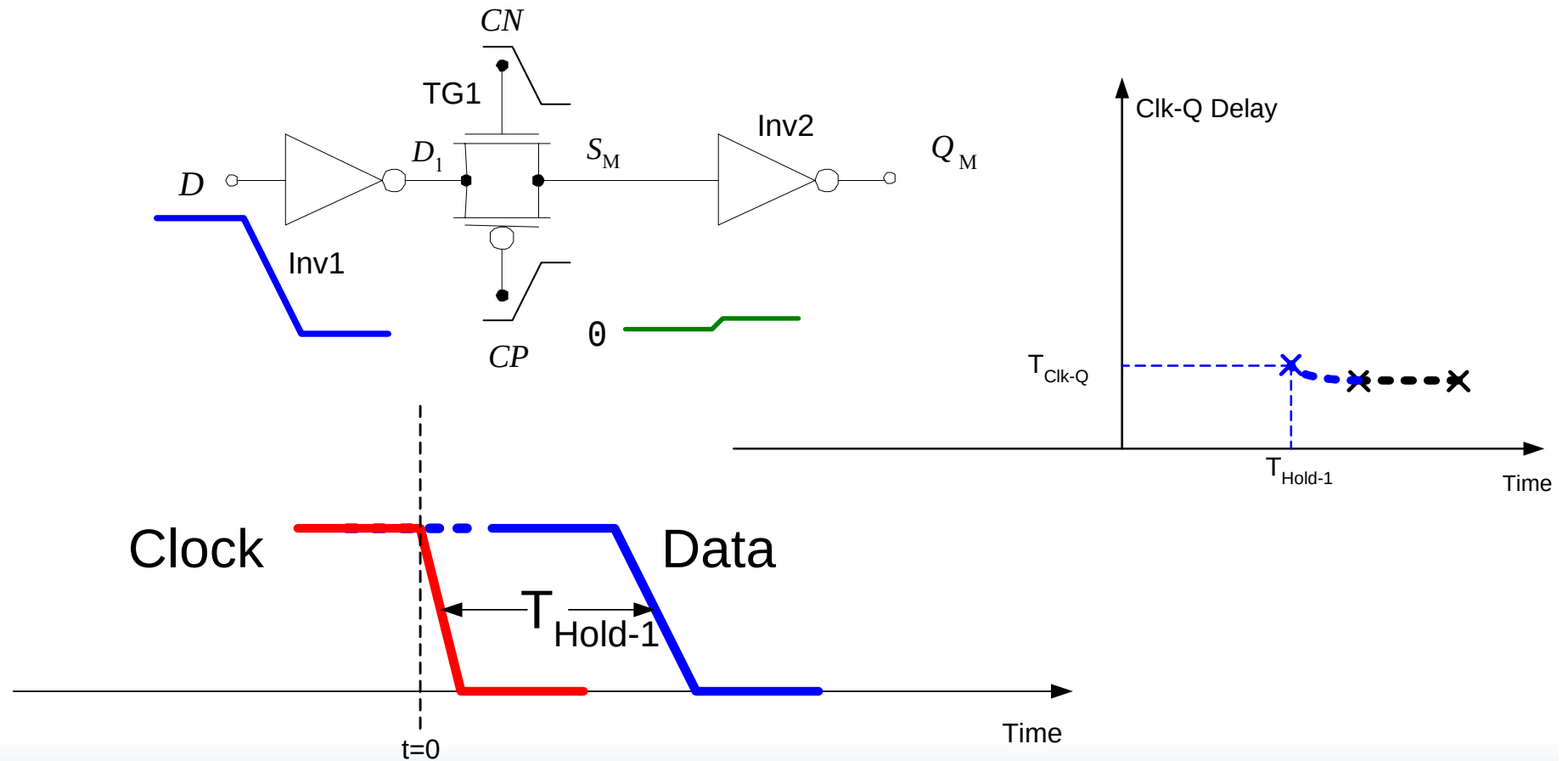
Setup/Hold Time Illustrations

Hold-1 case



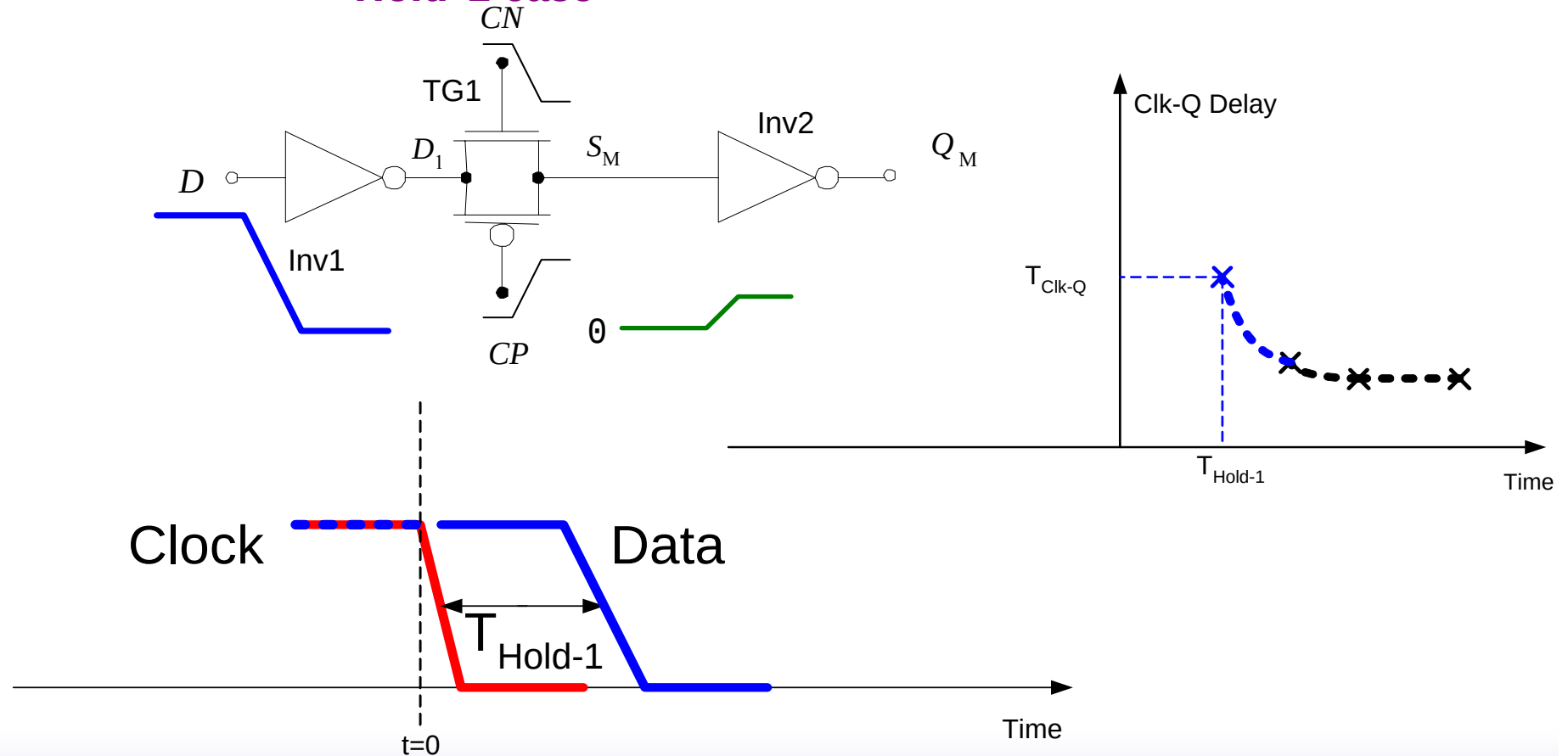
Setup/Hold Time Illustrations

Hold-1 case



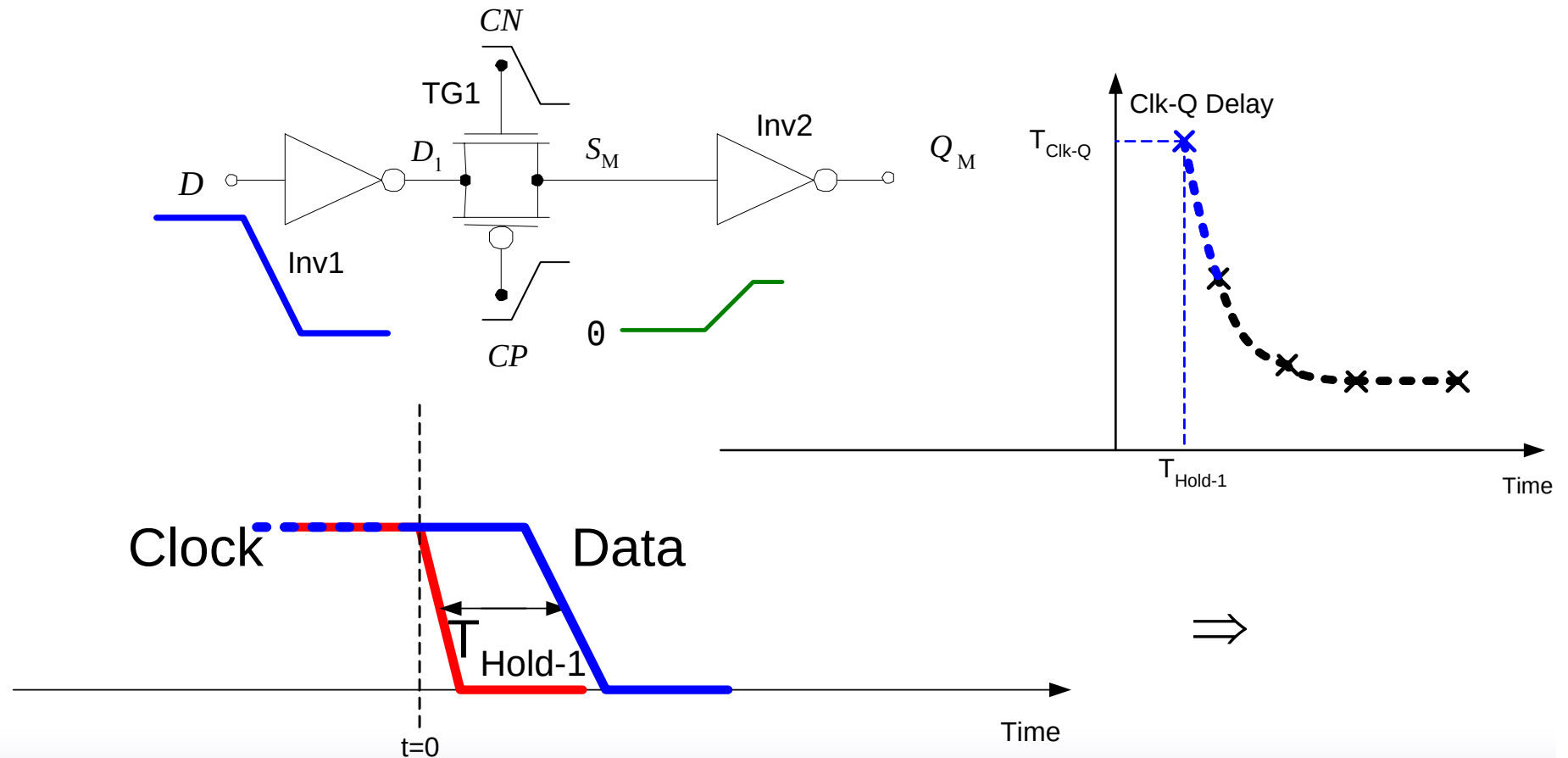
Setup/Hold Time Illustrations

Hold-1 case

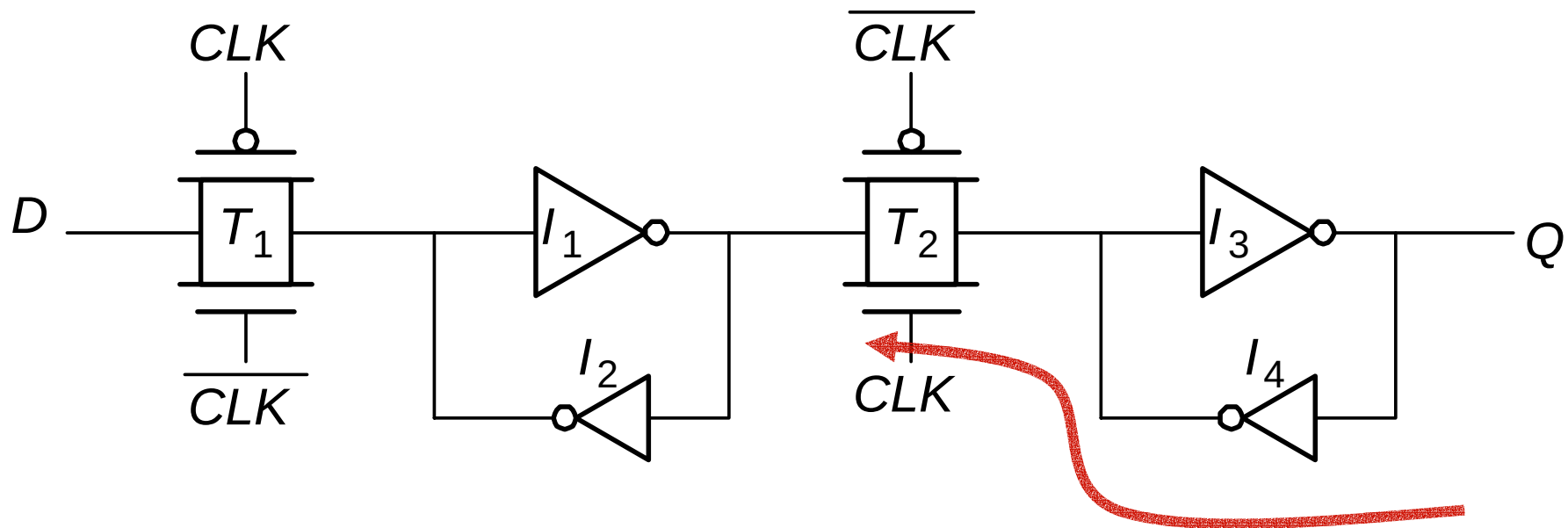


Setup/Hold Time Illustrations

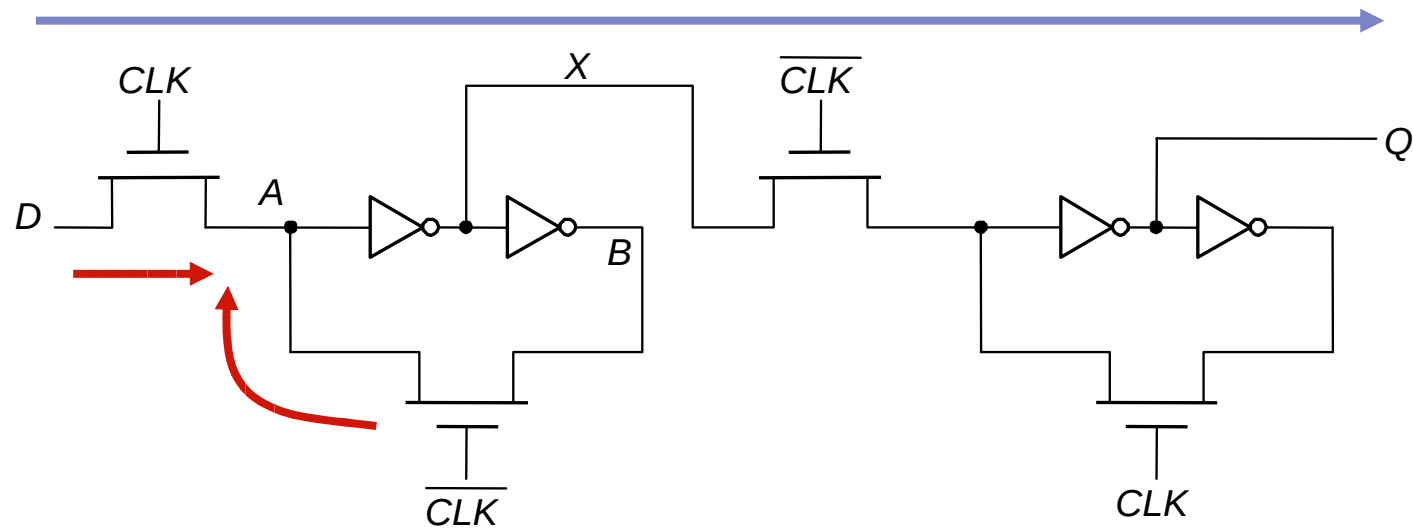
Hold-1 case



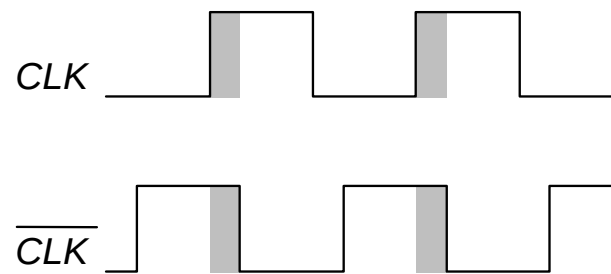
Reduced Clock Load Master-Slave Register



Avoiding Clock Overlap



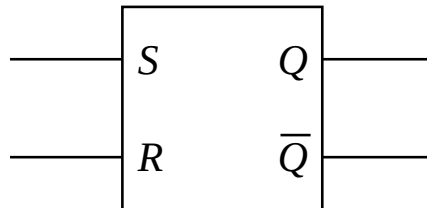
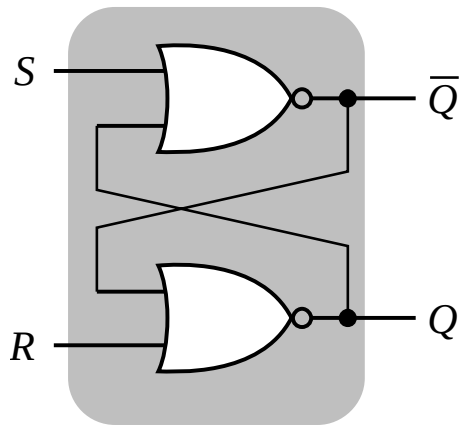
(a) Schematic diagram



(b) Overlapping clock pairs

Cross-coupled NOR Pair

NOR-based set-reset

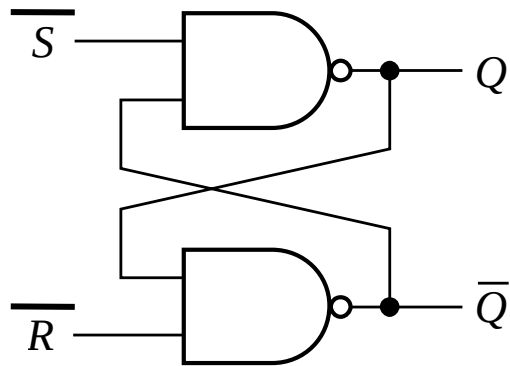


S	R	Q	$\bar{Q}$
0	0	Q	$\bar{Q}$
1	0	1	0
0	1	0	1
1	1	0	0

Forbidden State

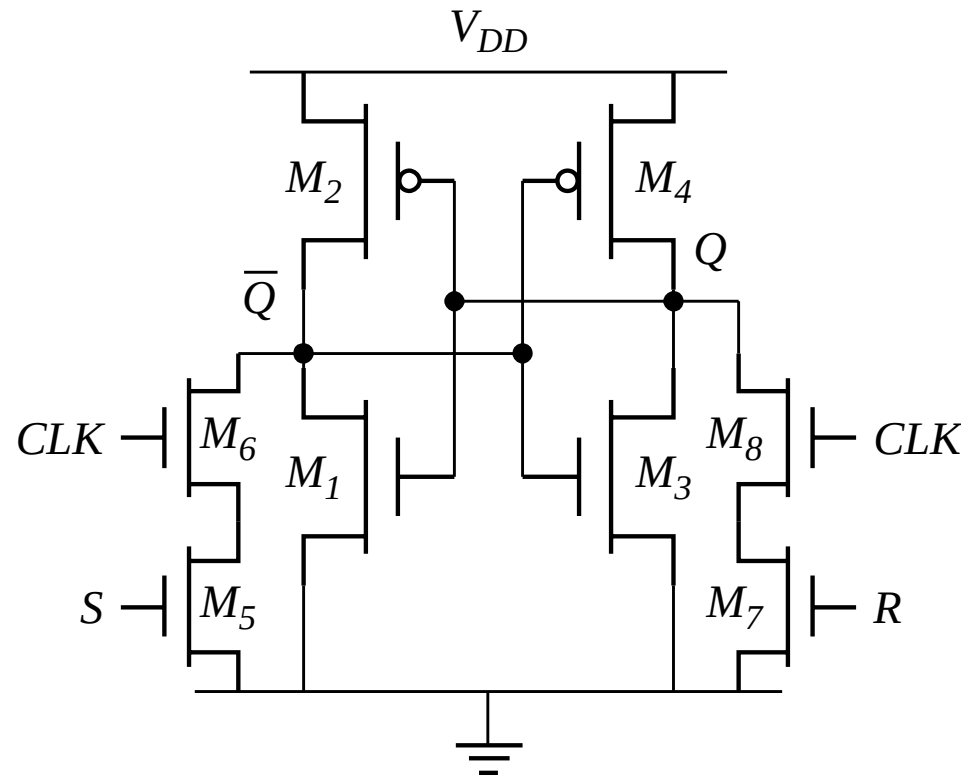
Cross-coupled NAND pair

Cross-coupled NANDs



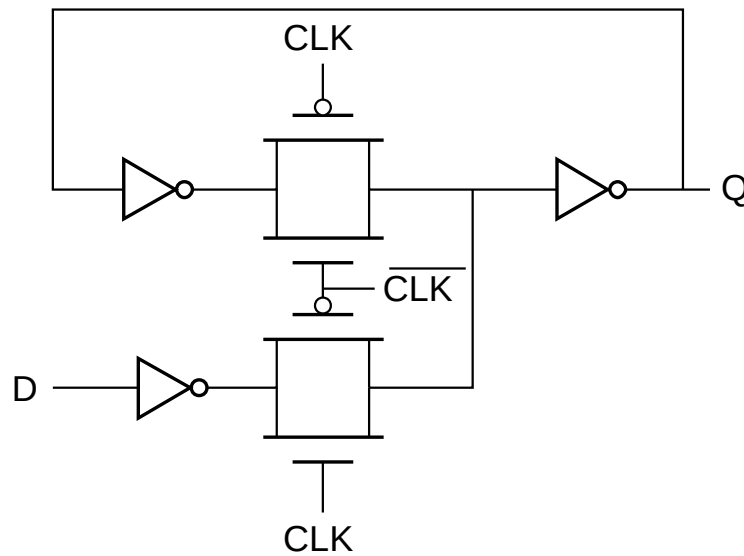
$\overline{S}$	$\overline{R}$	Q_n	$\overline{Q_n}$
0	0	N/A	N/A
0	1	1	0
1	0	0	1
1	1	Q_{n-1}	$\overline{Q_{n-1}}$

Set-Reset NOR Latch

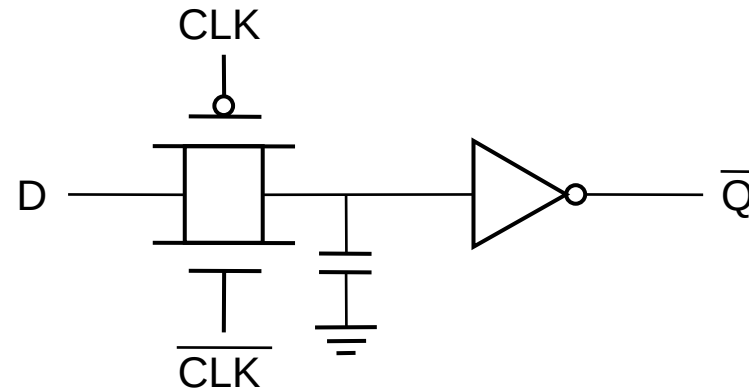


This is not used in datapaths any more,
but is a basic building memory cell

Storage Mechanisms

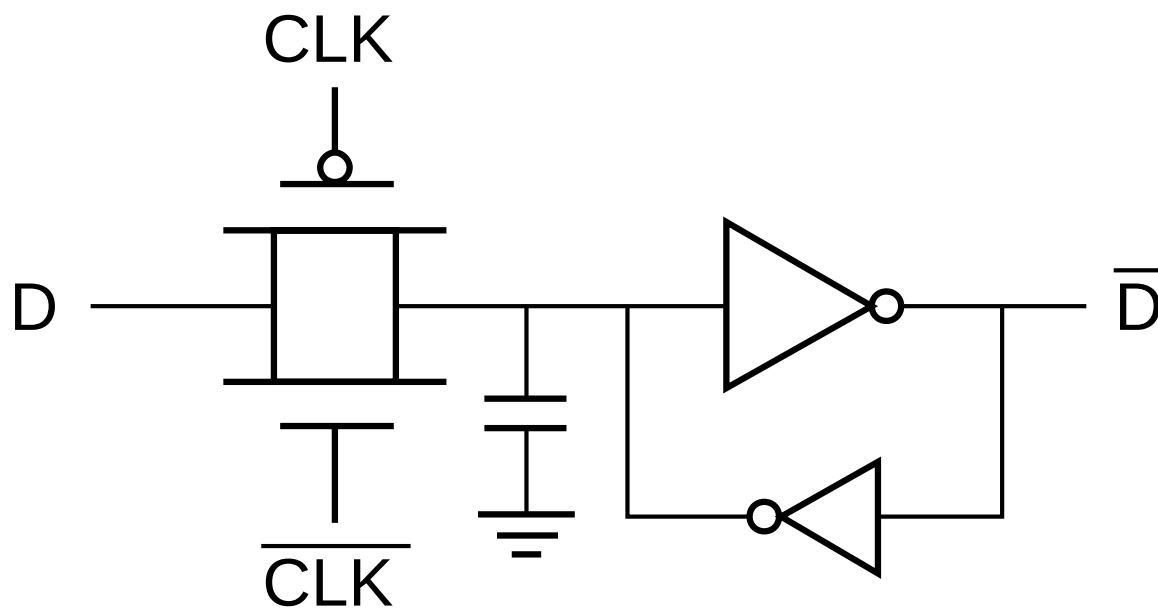


Static

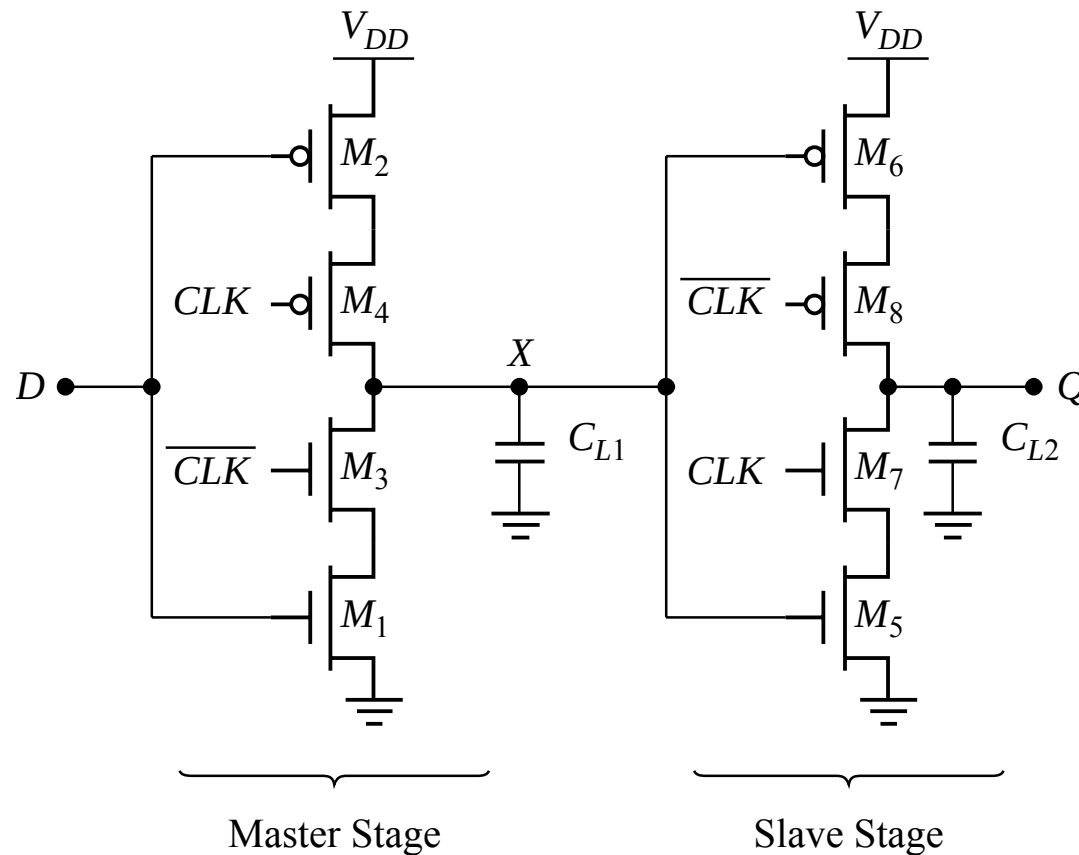


Dynamic (charge-based)

Making a Dynamic Latch Pseudo-Static

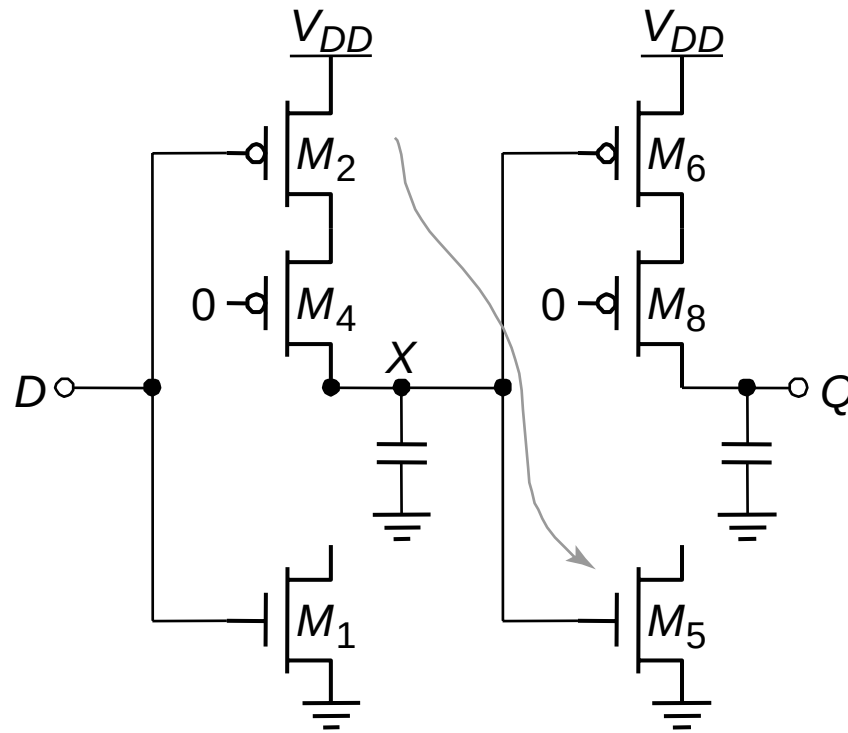


Dynamic Latches/Registers: C²MOS

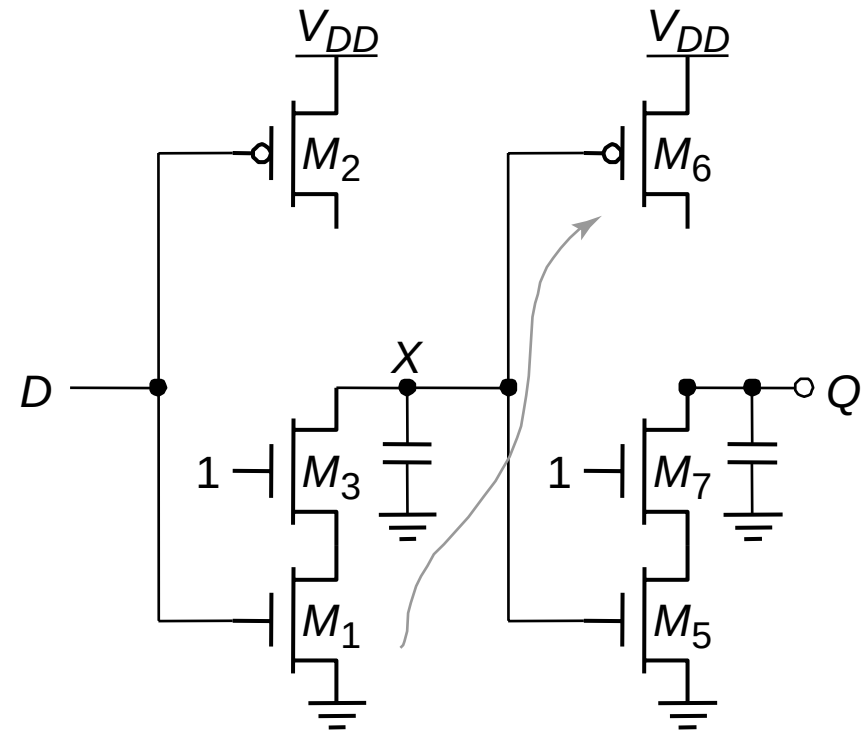


“Keepers” can be added to make circuit pseudo-static

Insensitive to Clock-Overlap

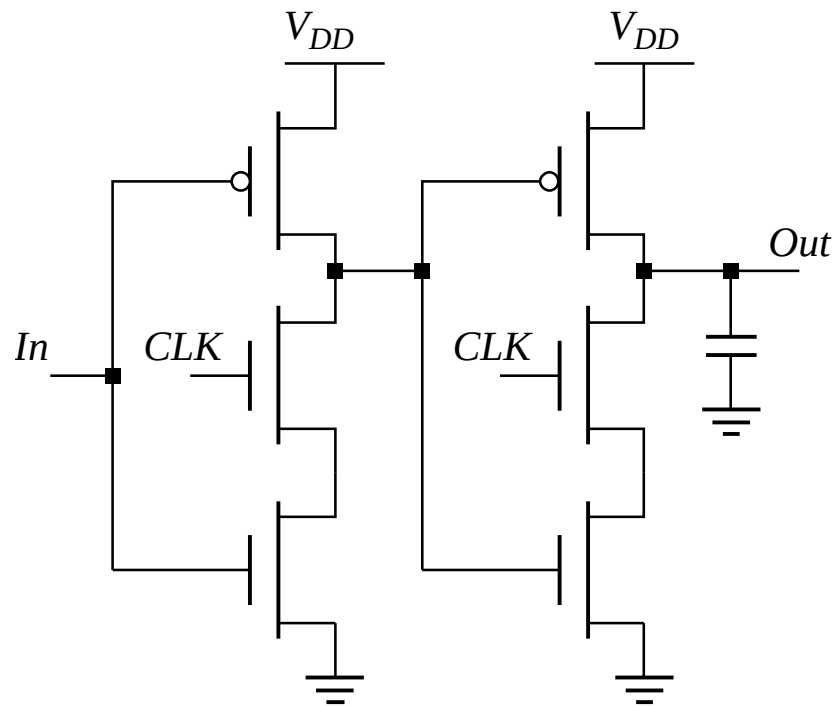


(a) (0-0) overlap

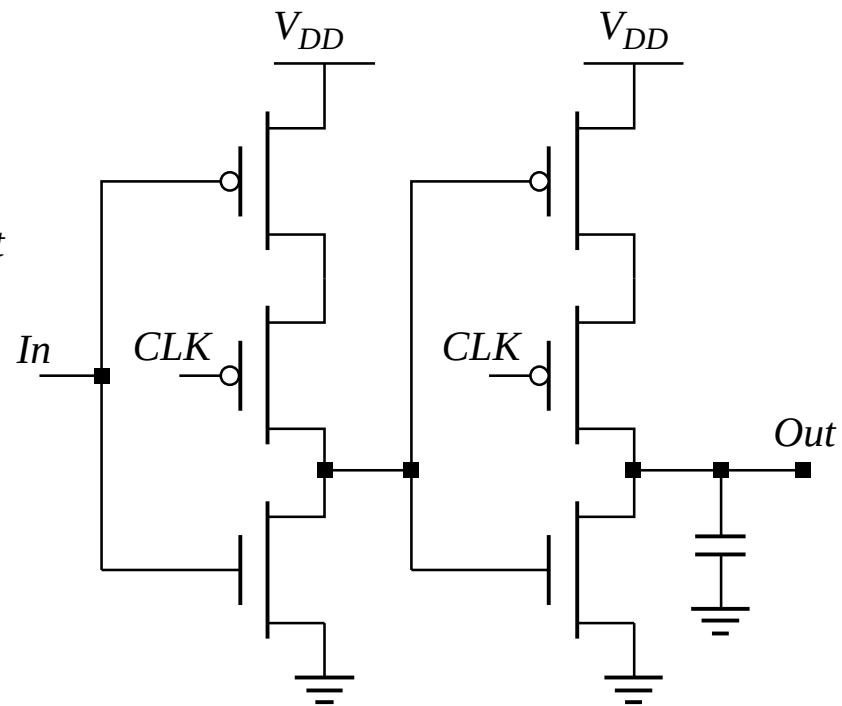


(b) (1-1) overlap

Single-phase Latches: TSPC

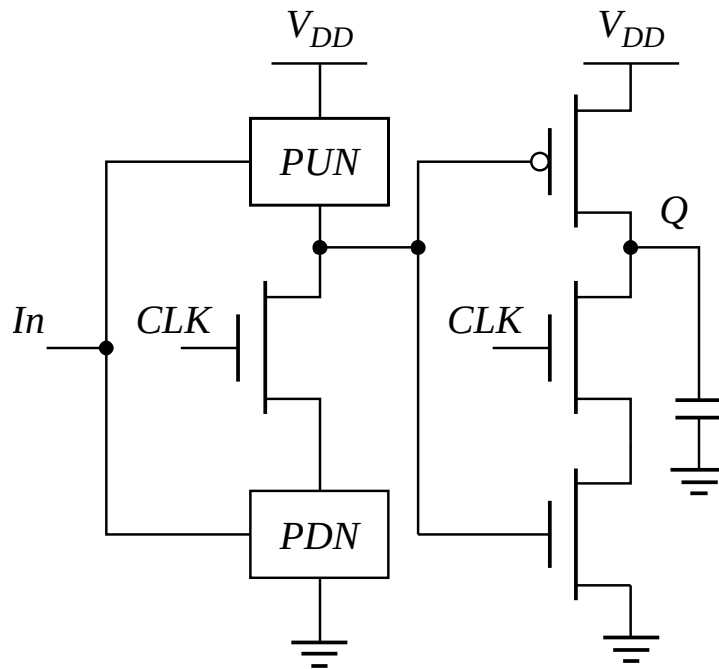


Double n-C²MOS

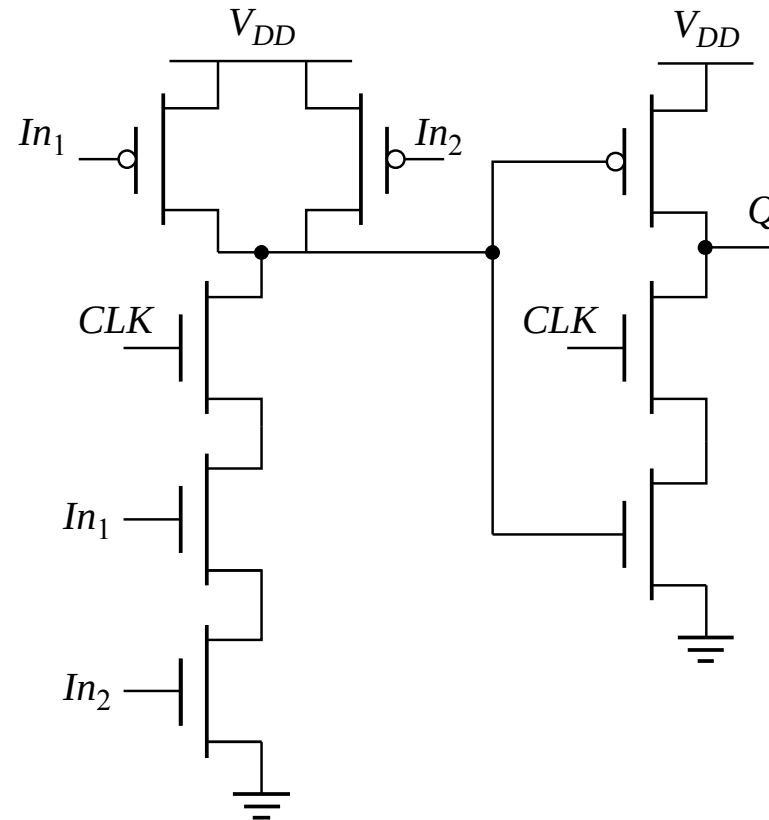


Double p-C²MOS

Including Logic in TSPC

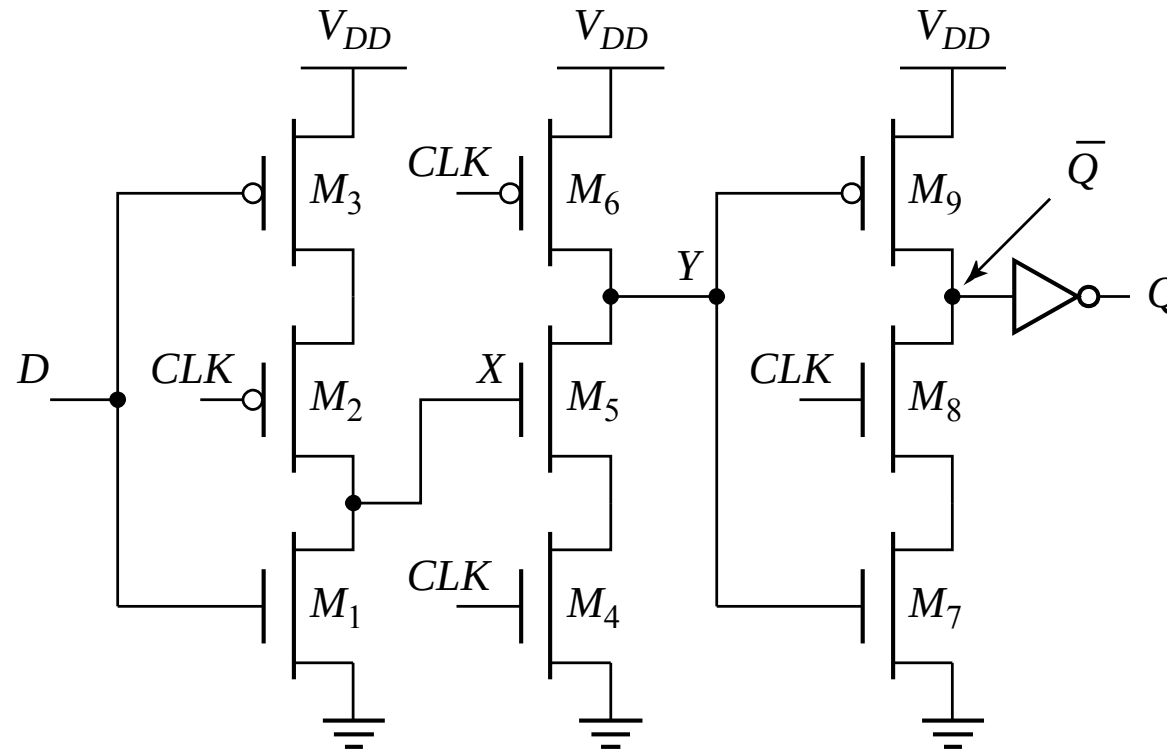


Example: logic inside the latch



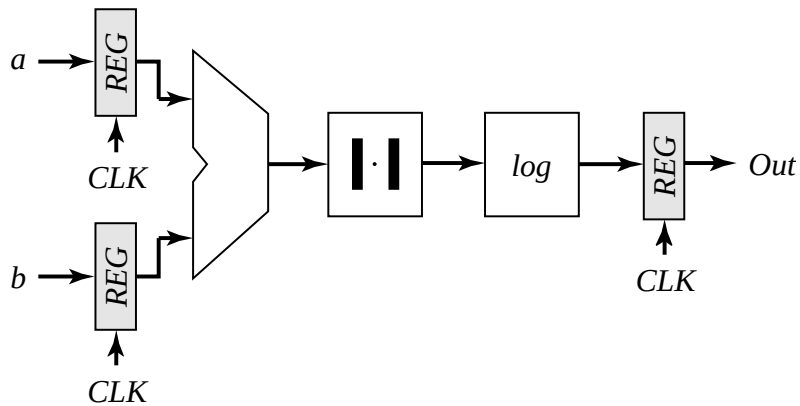
AND latch

Precharged Register (TSPC)

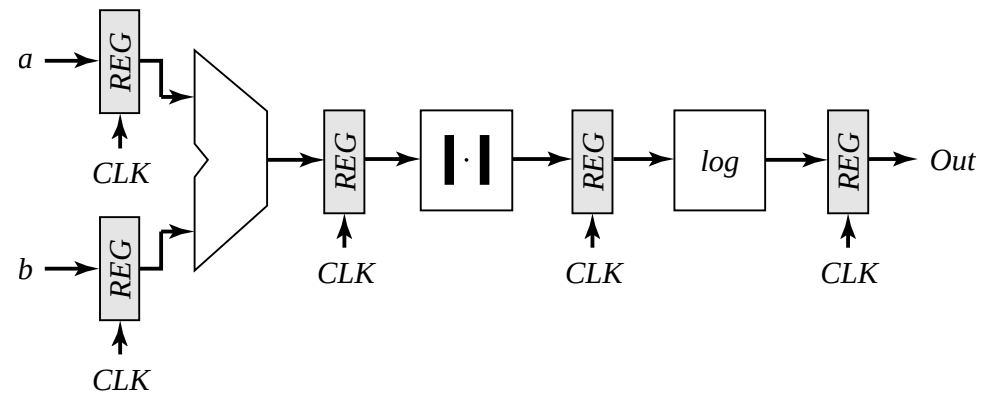


p-C²MOS – double n-C²MOS

Pipelining



Reference



Pipelined

Clock Period	Adder	Absolute Value	Logarithm
1	$a_1 + b_1$		
2	$a_2 + b_2$	$ a_1 + b_1 $	
3	$a_3 + b_3$	$ a_2 + b_2 $	$\log(a_1 + b_1)$
4	$a_4 + b_4$	$ a_3 + b_3 $	$\log(a_2 + b_2)$
5	$a_5 + b_5$	$ a_4 + b_4 $	$\log(a_3 + b_3)$